



Diplomová práce

Local-first přístup k vývoji webových aplikací

Studijní program:

N0613A140028 Informační technologie

Autor práce:

Bc. Adam Petříček

Vedoucí práce:

Ing. Jana Vitvarová, Ph.D.

Ústav mechatroniky a technické informatiky

Liberec 2026



Zadání diplomové práce

Local-first přístup k vývoji webových aplikací

Jméno a příjmení:

Bc. Adam Petříček

Osobní číslo:

M24000143

Studijní program:

N0613A140028 Informační technologie

Zadávající katedra:

Ústav mechatroniky a technické informatiky

Akademický rok:

2025/2026

Zásady pro vypracování:

1. Seznamte se s přístupem local-first a porovnejte jej s alternativními možnostmi tvorby webových aplikací.
2. Analyzujte dostupné knihovny a datové úložiště vhodné pro tvorbu local-first aplikací a popište jejich klíčové vlastnosti a zhodnoťte mechanismy synchronizace a bezpečnosti.
3. Navrhněte vhodné rozšíření u vybraných open-source local-first technologií (např. Evolu).
4. Navrhněte a implementujte ukázkovou aplikaci, která využívá local-first přístup a demonstruje jeho výhody.
5. Zdokumentujte metodiku návrhu a vývoje, průběh testování a způsob nasazení.

Rozsah grafických prací:	dle potřeby dokumentace
Rozsah pracovní zprávy:	40 až 50 stran
Forma zpracování práce:	tištěná/elektronická
Jazyk práce:	čeština

Seznam odborné literatury:

1. KLEPPMANN, Martin; WIGGINS, Adam; VAN HARDENBERG, Peter a MCGRANAGHAN, Mark. Local-first software: you own your data, in spite of the cloud. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* [online]. New York: Association for Computing Machinery. Onward! 2019. ISBN 978-1-4503-6995-4.
Dostupné z: <https://doi.org/10.1145/3359591.3359737>
2. KÖHLER, Mirko; ZAKHOUR, George; WEISENBURGER, Pascal a SALVANESCHI, Guido. Consistent Local-First Software: Enforcing Safety and Invariants for Local-First Applications. *IEEE Transactions on Software Engineering*. Vol: 51, No: 1, 2025.
Dostupné z: <https://doi.org/10.1109/TSE.2024.3477723>
3. GARAGUSO, Pablo David. *Vue.js 3 design patterns and best practices: develop scalable and robust applications with Vite, Pinia, and Vue Router*. Packt Publishing. 2023. ISBN 978-1-80324-900-1.

Vedoucí práce:	Ing. Jana Vitvarová, Ph.D. Ústav mechatroniky a technické informatiky
----------------	--

Datum zadání práce:	8. října 2025
Předpokládaný termín odevzdání:	11. května 2026

doc. Ing. Josef Černohorský, Ph.D.
děkan

L.S.

doc. Ing. Petr Červa, Ph.D.
garant studijního programu

Prohlášení

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.

Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.

Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.

Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitou v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.

Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS/STAG se shoduje s textem tištěné podoby práce.

Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

Poděkování

Rád bych poděkoval vedoucí této práce Ing. Janě Vitvarové, Ph.D. za poskytnuté rady a Danielu Steigerwaldovi za vytvoření knihovny Evolu a konzultaci problematiky.

Local-first přístup k vývoji webových aplikací

Abstrakt

Tato diplomová práce se zabývá analýzou local-first přístupu, vybraných technologií pro jeho implementaci a vytvořením ukázkové webové aplikace. Local-first přístup považuje lokální kopii dat za primární zdroj pravdy, díky čemuž aplikace funguje plně offline a uživatel vlastní svá data. Rešerše popisuje hlavní vlastnosti tohoto přístupu, důsledky pro uživatele a vývojáře, technologie pro ukládání a synchronizaci dat, správu uživatelské identity a bezpečnostní hrozby. Dále je proveden přehled ekosystému se zaměřením na knihovnu Evolu, jejíž vnitřní implementace je podrobně popsána. Praktická část zahrnuje open-source příspěvky do Evolu a ukázkovou aplikaci ve Vue pro evidenci výpůjček laboratorního vybavení.

Klíčová slova: Local-first, CRDT, synchronizace dat, E2E šifrování, SQLite, OPFS, Vue.js, TypeScript

Local-first approach for web application development

Abstract

This master's thesis analyzes the local-first approach, selected implementation technologies, and develops a sample web application. The local-first approach treats the local data copy as the primary source of truth, allowing the application to function fully offline while users retain ownership of their data. The research part describes its main characteristics, implications for users and developers, technologies for data storage and synchronization, user identity management, and security threats. It also provides an ecosystem overview focused on the Evolu library, including a detailed description of its internal implementation. The practical part includes open-source contributions to Evolu and a sample Vue application for tracking laboratory equipment borrowing.

Keywords: Local-first, CRDT, data synchronization, E2E encryption, SQLite, OPFS, Vue.js, TypeScript

Obsah

Seznam zkratek	11
1 Úvod	12
2 Local-first software	13
2.1 Motivace	13
2.2 Definice pojmů	14
2.3 Klíčové vlastnosti	15
2.4 Důsledky pro uživatele	18
2.5 Důsledky pro vývojáře	19
2.6 Problémy a výzvy	20
2.7 Oblasti využití	21
2.8 Historický kontext	23
2.9 Budoucí vývoj	24
3 Technologie a architektura	26
3.1 Ukládání dat	26
3.1.1 SQLite	26
3.1.2 Vývoj webových technologií	27
3.1.3 Origin Private File System	29
3.1.4 Web Workers	30
3.2 Synchronizace dat	30
3.2.1 Distribuované systémy	31
3.2.2 Conflict-free Replicated Data Types	32
3.2.3 Hybrid Logical Clocks	33
3.2.4 Range Based Set Reconciliation	34
3.3 Šifrování dat a správa klíčů	35
3.3.1 Historie šifrování	36
3.3.2 Identita uživatele	37
3.3.3 Passkeys a WebAuthn	38
3.4 Uživatelská kolaborace	39
3.4.1 Členství ve skupině uživatelů	40
3.4.2 Message Layer Security	40
3.5 Bezpečnostní hrozby	41
3.5.1 Meltdown a Spectre	42

4	Local-first knihovny	44
4.1	Přehled ekosystému	44
4.2	Knihovna Evolu	45
4.3	Vnitřní implementace Evolu	45
4.3.1	TypeScript knihovna	46
4.3.2	Ukládání dat	47
4.3.3	Přenos dat	48
4.3.4	Relay	49
4.3.5	Owner API	49
4.3.6	Bezpečnost a soukromí	50
4.4	Open-source rozšíření Evolu	51
5	Implementace local-first aplikace	53
5.1	Obecná struktura	53
5.2	Zaměření ukázkové aplikace	54
5.3	Použité technologie	54
5.4	Datové schéma	55
5.5	Popis funkcí	55
5.6	Příklad použití v praxi	57
5.7	Testování	58
5.8	Build a deployment	58
6	Závěr	59
	Seznam použité literatury	60
A	Odkaz na zdrojový kód	66

Seznam obrázků

2.1	Porovnání přístupů k architektuře aplikace	14
3.1	Datová struktura Skip List	35
4.1	Layer-architektura knihovny Evolu	45
4.2	Struktura zprávy v Evolu Protocol [78]	49
4.3	Sutnar Identicon po úpravě kontrastu	52
5.1	Domovská stránka ukázkové aplikace a stránka s nastavením	56
5.2	Detail místnosti s přehledem dostupných a vypůjčených předmětů a historií operací	56
5.3	Přidání místnosti s možností připojit se k existující nebo vytvoření nové	57

Seznam tabulek

2.1	Porovnání přístupů k architektuře aplikace	18
3.1	Porovnání technologií pro persistenci dat v prohlížeči	29

Seznam zkratek

AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
AWS	Amazon Web Services
BIP	Bitcoin Improvement Proposal
CRDT	Conflict-free Replicated Data Type
CTAP2	Client to Authenticator Protocol 2
CTE	Common Table Expression
DNS	Domain Name System
DOM	Document Object Model
FIDO2	Fast IDentity Online 2
HATEOAS	Hypermedia as the Engine of Application State
HLC	Hybrid Logical Clock
IETF	Internet Engineering Task Force
LWW	Last Write Wins
MLS	Message Layer Security
NAT	Network Address Translation
NFC	Near Field Communication
OPFS	Origin Private File System
PRF	Pseudo-Random Function
PWA	Progressive Web App
RBSR	Range-Based Set Reconciliation
REST	Representational State Transfer
RFC	Request for Comments
SLIP	Satoshi Labs Improvement Proposal
SPA	Single Page Application
SSL	Secure Sockets Layer
TLS	Transport Layer Security
TURN	Traversal Using Relays around NAT
W3C	World Wide Web Consortium
WAL	Write-Ahead Logging
WebRTC	Web Real-Time Communication
XSS	Cross-Site Scripting

1 Úvod

Většina dnešního softwaru je tvořena cloudovými aplikacemi, kde uživatel přes webový prohlížeč přistupuje k datům uloženým na serverech poskytovatele. Tento model přináší možnost snadné uživatelské kolaborace a synchronizace mezi zařízeními, ale zároveň znamená ztrátu vlastnictví dat ze strany uživatele a závislost aplikace na neustálém připojení k internetu. Nabízí se otázka, zda lze vyřešit tyto nedostatky a přitom si zachovat uvedené výhody. Jednou z možností je tzv. local-first přístup.

Rešeršní část této práce začíná kapitolou 2, kde je local-first přístup nejprve popsán obecně – motivace pro jeho vznik, ucelená definice, sedm klíčových vlastností a porovnání s ostatními přístupy. Dále jsou rozebrány důsledky pro uživatele i vývojáře, možné problémy, historický kontext a možná budoucí směřování. Rešeršní část pokračuje kapitolou 3, která se věnuje konkrétním technologiím pro tvorbu local-first aplikací – ukládání dat v prohlížeči, synchronizační algoritmy, správa kryptografických klíčů a identity uživatele, uživatelská kolaborace a bezpečnostní hrozby vyplývající z přesunu dat a logiky na klienta.

Další částí práce je přehled local-first knihoven nacházející se v kapitole 4. Obsahuje rozdělení ekosystému do čtyř kategorií dle úrovně abstrakce a poté detailní popis vnitřní implementace knihovny Evolu. Praktická část práce dále popisuje open-source příspěvky do knihovny Evolu a v kapitole 5 implementaci ukázkové aplikace. Nejprve je nastíněna obecná struktura local-first aplikace, poté je popsáno konkrétní zaměření ukázkové aplikace, použité technologie a funkcionality. Závěr kapitoly popisuje testování a způsob nasazení aplikace.

2 Local-first software

2.1 Motivace

Lidé během svého života produkují velké množství různorodých dat. Jedná se např. o dokumenty, tabulky, obrázky, zdrojový kód, hesla, zdravotní dokumentaci, poznámky v kalendáři nebo zprávy v konverzacích. Data často obsahují citlivé údaje, případně jsou výsledkem různých kreativních činností člověka. Lze předpokládat, že se lidé k těmto datům budou chtít v budoucnu vracet a dále s nimi pracovat nebo je archivovat. [1]

V nynější době lidé pro práci s těmito daty typicky používají cloudové aplikace. Ty jsou charakterizovány centralizovaným uložením dat na serveru, ke kterým lze přistupovat přes webový prohlížeč nebo mobilní aplikaci. To sice poskytuje možnost snadno kolaborovat s dalšími uživateli a dostupnost dat na jiných zařízeních, ale cenou za to je **ztráta vlastnictví dat**. [2]

Přestože data produkuje uživatel, fyzicky jsou uložena na serverech poskytovatele – prakticky tedy data vlastní právě on. Uživatel je závislý na tom, že mu poskytovatel umožní přístup k datům (dle jeho podmínek, které se mohou kdykoliv změnit). Kvůli centralizaci dat na serveru je pro používání aplikace vyžadováno neustálé připojení k internetu. Každá operace s daty tak nutně zahrnuje síťovou komunikaci, čímž se nutně přidá latence, která zpomaluje odezvu uživatelského rozhraní a z principu se jí nelze zbavit. [3]

Další z problémů může nastat, pokud data centralizujeme na infrastruktuře malého počtu globálních poskytovatelů. Příkladem může být Cloudflare, jehož infrastruktura obsluhuje přibližně pětinu veškerého webového provozu [4]. Jen během čtvrtého kvartálu roku 2025 nastaly dva rozsáhlé výpadky Cloudflare, které kompletně znemožnily přístup ke všem aplikacím využívajícím Cloudflare [5] [6]. Konektivita je tak jedním z kritických bodů selhání u cloudového software – a aplikace, které neukládají žádný stav lokálně, se v takových chvílích stávají zcela nepoužitelnými, přestože data stále existují.

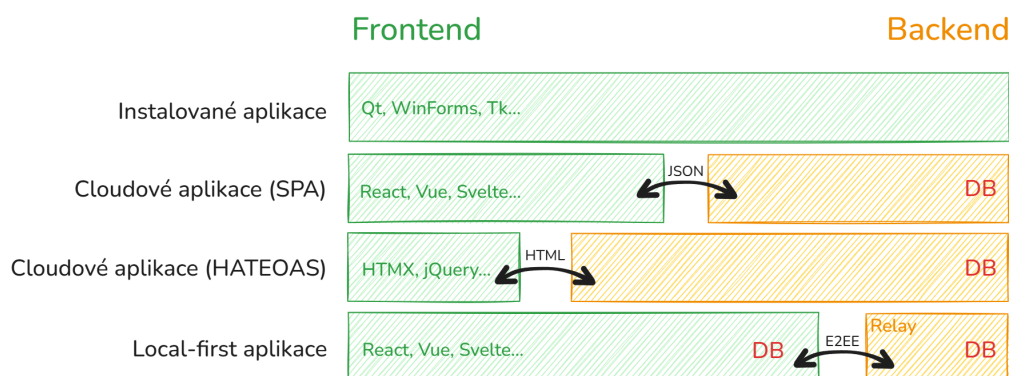
Pokud data vlastní poskytovatel služby, může je zároveň sdílet s libovolnou třetí stranou. V minulosti došlo k různým kauzám jako např. Cambridge Analytica (2018), kdy byla získána osobní data více než 87 milionů uživatelů služby Facebook. Prostředkem k získání dat byl osobnostní kvíz, který vyplnilo 270 000 lidí. Kvůli tehdejšímu nastavení API byl ovšem umožněn také přístup k datům všech jejich přátel.

2.2 Definice pojmů

V první řadě je nutné vymezit dva koncepty, ze kterých local-first vychází. Tzv. **instalované aplikace** pracují výhradně s lokálními daty uloženými na disku uživatele. Jelikož tyto aplikace nikdy nekomunikují se vzdáleným serverem, internetové připojení není pro jejich funkčnost vyžadováno. Lze tak označit v podstatě veškerý software před nástupem internetu. Typická je instalace z datového nosiče (CD, disketa), který byl zakoupen v kamenném obchodě. Tyto aplikace lze také označit jako „local-only”. Jako příklad lze uvést starší verze aplikací Adobe Photoshop nebo Microsoft Word. Na opačném konci spektra stojí tzv. **cloudové aplikace**, které považují vzdálený server za zdroj pravdy. Data jsou uložena centralizovaně na serveru a funkčnost aplikace je plně závislá na stabilním internetovém připojení. Na druhou stranu existuje možnost data synchronizovat mezi více zařízení a kolaborovat nad nimi s dalšími uživateli. Jako příklad lze uvést aplikace Google Docs, Figma či Trello.

Za local-first lze považovat takovou aplikaci, která si udržuje lokální kopii dat a považuje ji za primární zdroj pravdy. Synchronizace dat mezi zařízeními probíhá na pozadí, pokud je dostupné internetové připojení – jinak aplikace funguje plně offline.

Local-first je tedy spojením staršího konceptu klasických instalovaných aplikací a novějšího konceptu cloudových aplikací. Snaží se vzít si z obou světů ty dobré vlastnosti – z instalovaných aplikací možnost vlastnit data díky uložení v lokální databázi a z cloudových aplikací možnost k datům přistupovat odkudkoliv. Poprvé byl tento pojem použit v esejí *Local-first software: You own your data, in spite of the cloud*, kterou v roce 2019 publikovala výzkumná skupina Ink & Switch.



Obrázek 2.1: Porovnání přístupů k architektuře aplikace

Existuje více způsobů, jak lze na local-first pohlížet. Martin Kleppmann například tvrdí, že by (ne)dostupnost jiného počítače neměla ovlivnit schopnost uživatele manipulovat s daty. „Pokud vývojář aplikace přeruší vývoj, vypne servery a aplikace přestane fungovat, potom není local-first.” Další z možných pohledů je, že local-first je útokem na všechny formy proprietárního uzamčení (vendor lock-in) u protokolů, datových formátů, služeb atd. [7]

Offline-first

Kromě local-first se lze setkat také s pojmem **offline-first**. Ten poprvé použil Alex Feyerke v článku *Designing Offline-First Web Apps* z roku 2013. Hlavní myšlenkou bylo, aby se při vývoji aplikací již od počátku počítalo s tím, že má aplikace fungovat i bez internetu. Jedná se o pojem méně striktní než local-first a lze říci, že local-first je nadmnožinou offline-first. Také je nutné poznamenat, že offline-first nemá pevně ustálenou definici – toto je pouze pokus o co nejlepší interpretaci v kontextu.

V praxi může být offline-first často označením pouze pro to, že si klasická klient-server aplikace umí lokálně cachovat data v případě nedostupnosti serveru. Jakmile je server opět dostupný, uložený stav se tam zpět nahraje a ze zařízení ho lze smazat. Rozdíl je tedy v tom, zda je pro aplikaci lokální kopie dat zdrojem pravdy.

Pojmy local-first a offline-first mohou být někdy zaměňovány, ale odlišují se především motivací, která stála za jejich vznikem. Offline-first řeší spíše technický problém: „Co se stane, když přestane fungovat internet?“ a local-first spíše filozofický problém: „Kdo v aplikaci vlastní uživatelská data a jak je s nimi schopen nakládat?“ [8] [9]

2.3 Klíčové vlastnosti

Je nutno si uvědomit, že local-first není binární pojem, ale spíše škála. Jedná se o soubor principů pro kvalitnější software. V již zmiňované eseji od Ink & Switch bylo definováno sedm ideálů pro local-first software [1]. Martin Kleppmann popisuje, že se nejedná o striktní definici, ale spíše o směr, kterým by se měl ekosystém vyvíjet. Pokud aplikace nesplňuje všechny body, nutně to tedy ještě neznamená, že není local-first. První čtyři body sdílejí průnik s konceptem offline-first a jsou obecně snáze dosažitelné, zatímco poslední tři body jsou specifické pro local-first.

I přes tyto relativně striktní požadavky existují aplikace/knihovny, které se nazývají local-first, přestože používají např. autoritativní centrální server. Ekosystém se neustále vyvíjí a být čistě local-first nemusí být pro každý typ aplikace nejlepší volbou. Vyřešit všechny problémy může trvat roky a bezesporu existuje prostor i pro částečná řešení. Local-first přístup také nezmiňuje konkrétní technologie ani způsob implementace – může se tedy jednat o aplikaci určenou ať už pro webový prohlížeč, nativního desktopového či mobilního klienta (Windows, macOS, Android, iOS) nebo například embedded zařízení (Raspberry Pi).

1) Reaktivní UI s téměř instantní odezvou, minimum načítacích koleček

Operace čtení i zápisu probíhají proti lokální databázi, nemusí tudíž čekat na odpověď vzdáleného serveru a odezva lokální databáze je řádově v jednotkách milisekund. Díky tomu může být uživatelské rozhraní reaktivní – libovolné změny v databázi se automaticky promítnou do zobrazení. Formulace „minimum načítacích koleček“ neznamená, že se indikátor nikdy nesmí zobrazit – při déle trvajících akcích, jako např. počáteční synchronizaci velkého množství dat, je zcela na místě.

Důraz je hlavně na to, že běžné operace jako vyhledávání, editace nebo stránkování by měly z pohledu uživatele působit jako instantní. V praxi to tedy znamená, že po uživatelské interakci chceme překreslit rozhraní ideálně hned v dalším snímku (pod 16 ms).

2) Data nejsou uzavřena v rámci jednoho zařízení

Úpravy provedené na jednom zařízení jsou automaticky dostupné i na ostatních zařízeních uživatele. Synchronizace probíhá na pozadí a ideálně nevyžaduje žádný manuální zásah – uživatel nemusí ručně kopírovat soubory ani exportovat data. Vychází z myšlenky, že není nutné psát synchronizační vrstvu pro každou aplikaci zvlášť, ale lze ji napsat genericky. Z toho vyplývá také nutnost řešit všechny problémy spojené se synchronizací dat, jež jsou podrobněji popsány v kapitole 3.2.

3) Připojení k internetu není vyžadované

Klíčová je offline dostupnost všech dat patřících danému uživateli. Z toho plyne, že aplikace plně funguje i bez připojení k internetu. S tím se počítá od začátku vývoje, což je rozdíl oproti cloudovým aplikacím, kde se podpora pro offline mód typicky přidává až dodatečně. Kvůli tomu poté vznikají frustrující situace, jako například když se po provedené offline práci v Google Docs uživatel při opětovném připojení k internetu dozví, že serverový stav se mezitím příliš změnil a změny proto nelze sloučit. Tím je stav resetován na serverový a uživatel přijde o offline provedené změny [10]. Někdo by mohl namítnout, že konektivita se neustále zlepšuje a připojení k internetu bude brzy dostupné všude. Je ovšem nutné si uvědomit, že toto platí jen pro určité privilegované části světa – a ani tam není internet spolehlivý za všech okolností. Stále také existují relevantní důvody, proč pracovat offline (minimalizace rušivých vlivů, anonymita).

4) Kolaborace s dalšími uživateli

Kolaborace zpravidla znamená, že více uživatelů může současně upravovat sdílená data. Ať už se jedná o společnou editaci dokumentu nebo zobrazení pozice kurzoru ostatních uživatelů v reálném čase, tento bod patří z hlediska implementace k jednomu z nejtěžších. Zahrnuje celou řadu kompromisů – od volby strategie řešení konfliktů, přes oprávnění ve skupině uživatelů až po granularitu synchronizace. Této problematice se podrobněji věnuje kapitola 3.4. Cílem je umožnit podobně bezproblémovou kolaboraci jako mají cloudové aplikace, ale v local-first prostředí.

5) Dlouhověkost dat

Pokud nějaký software existuje po dobu mnoha let, jen zřídka je stále užitečný (samozřejmě existují výjimky), ovšem samotná data bývají často hodnotná stejně jako dříve – nebo i více. Local-first aplikace by měly umožnit pracovat s vytvořenými daty i za desítky let, ideálně bez silné závislosti na třetích stranách. To zahrnuje používání otevřených formátů, umožnění exportu dat a stabilitu datového

modelu. Typickým příkladem vhodného formátu dat je SQLite, jehož vývojáři počítají s podporou trvající minimálně do roku 2050 [11] a který americká Knihovna Kongresu uvádí jako doporučený archivační formát [12]. Současné cloudové aplikace trpí jevem, který lze nazvat jako „doba digitálního temna“ – jelikož funkcionality celé aplikace je neoddělitelně svázána se serverem poskytovatele, jakmile přestane fungovat, zanikají s ním i veškerá uživatelská data, čímž v určitém smyslu vzniká nevyplněná díra v historii softwaru [10]. Nejedná se přitom o hypotetické riziko: projekty jako Killed by Google dokumentují stovky služeb, které Google v průběhu let ukončil, přičemž uživatelé měli mnohdy jen několik týdnů na záchranu svých dat. V nejlepším případě poskytovatel nabídne alespoň export do JSON nebo CSV souboru – tím sice uživatel data získá zpět, ale přijde o veškerý kontext, historii změn a funkce aplikace. Poté je otázkou, zda vůbec ještě dokáže data využít.

6) Bezpečnost a anonymita

Problémem u cloudové architektury je uložení všech uživatelských dat centralizovaně, což vytváří cíl pro potenciální útočníky. V local-first modelu je proto server generický, díky čemuž se jako velmi příhodné nabízí implementovat end-to-end šifrování (E2EE) jako výchozí stav. Pokud jsou data zašifrována ještě na straně klienta, funkce serveru může být omezena na pouhé ukládání a přeposílání zašifrovaných blobů bez přístupu ke struktuře dat. To chrání v první řadě uživatele před neoprávněným přístupem k jejich datům a v druhé řadě poskytovatele služby, který nenese zodpovědnost za přenášený obsah. V podstatě lze na server pohlížet jen jako na další peer, který je neustále online.

7) Uživatel vlastní svá data a má možnost s nimi libovolně nakládat

Vlastnictví není v tomto kontextu myšleno jako právníkový koncept duševního vlastnictví, ale spíše ve smyslu uživatelské autonomie a kontroly nad daty. Uživatel by měl mít možnost s vlastními daty nakládat dle svého uvážení, libovolně je exportovat/importovat mezi aplikacemi, případně nad nimi mít možnost spouštět např. vlastní analytické skripty. Pokud jsou data uložena ve známém formátu, technicky zdatnější uživatelé s nimi mohou pracovat i mimo rozsah původní aplikace. S vlastnictvím dat souvisí i širší koncept tzv. „tvárnosti software“ (software malleability) – možnost si aplikaci přizpůsobit vlastním potřebám. Jedná se o případy, které jsou příliš specifické na to, aby byly zahrnuty do jádra původní aplikace. Příkladem mohou být rozšíření do webových prohlížečů, které provádějí drobné úpravy v CSS nebo automatizují opakované akce [13]. U softwaru je problém, že neexistuje dostatek obecných nástrojů – místo toho je velké množství rozdílných implementací, které spolu neumí spolupracovat [14]. Cílem celého tohoto bodu je, aby měl uživatel možnost maximálně naplnit svůj potenciál.

Porovnání přístupů

Pokud potřebujeme u cloudové aplikace přidat podporu pro offline režim, typicky se to řeší přidáním frontendového cachování a optimistic updates. Čím více

cachování přidáváme, tím více v podstatě budujeme druhou databázi na klientovi. Local-first tento stav přiznává a dotahuje do důsledků – lokální databáze už není pouze cache, ale primární zdroj pravdy. Zároveň local-first dělá most mezi nativními a webovými aplikacemi – web sice používáme jako platformu pro distribuci, ale data i celá aplikace se nachází na klientovi, díky čemuž ji dává smysl instalovat jako PWA, což už se více přibližuje nativní aplikaci.

Tabulka 2.1: Porovnání přístupů k architektuře aplikace

	Instalované	Cloudové	Local-first
Zdroj pravdy	Lokální úložiště	Centrální server	Lokální úložiště
Funkčnost bez internetu (read/write)	Plná	Omezená nebo žádná (dle cache)	Plná pro lokální data
Latence běžných operací	Velmi nízká	Závislá na síti a serveru	Velmi nízká (lokální operace)
Kolaborace v reálném čase	Žádná	Jednoduchá implementace	Možná, ale složitější implementace
Synchronizace dat mezi zařízeními	Ruční kopírování souborů	Vestavěná přes centrální server	Vestavěná přes generický sync engine
Vlastnictví dat	Uživatel	Poskytovatel služby	Uživatel
Soukromí a E2EE	Možné, ale ne vždy standard	Často omezené serverovým modelem	Přirozeně kompatibilní s E2EE modelem
Závislost na poskytovateli služby	Nízká	Vysoká	Nízká až střední (dle sync infrastruktury)
Provozní náročnost backendu	Žádná	Vysoká (DB, API, ops)	Střední (sync vrstva)
Automatické aktualizace	Ne	Ano	Ano
Dlouhověkost dat	Vysoká	Riziko při ukončení služby	Vysoká

2.4 Důsledky pro uživatele

Architektura local-first přináší pro koncové uživatele řadu praktických důsledků. Ten nejpodstatnější je, že **uživatel svá data skutečně vlastní**, má je uložené lokálně a má nad nimi plnou kontrolu. Poskytovatel aplikace mu nemůže upřít přístup k jeho vlastním datům – ať už na základě vědomého zabanování nebo nechtěného smazání

dat v důsledku chyby. Data uživatele také nemůže zneužít bez jeho vědomí, například pomocí prodeje třetím stranám za účelem cílené reklamy. Uživatel má nyní sice kontrolu nad svými daty, ale z toho plyne také zodpovědnost za jejich ztrátu (více rozepsané v kapitole 3.3.2).

Díky **důrazu na offline fungování** jsou local-first aplikace vhodné pro všechny uživatele s horším přístupem ke stabilnímu internetovému připojení. Může se jednat např. o lidi z rozvojových regionů (Afrika, Asie, Blízký východ), pro které představuje závislost na neustálém připojení zásadní bariéru. Problém s nestabilním připojením ovšem nastává i ve vyspělých zemích – stačí se nacházet ve vlaku, který projíždí tunelem nebo v odlehlejších či horských oblastech. S tím souvisí také uživatelé pracující v terénu – např. farmáři, zdravotníci, novináři, piloti nebo stavební inženýři.

Díky tomu, že local-first podporuje synchronizační služby, které jsou genericke a nezávislé na aplikaci, pro uživatele je **jednodušší změnit poskytovatele backendu**. V případě, že se poskytovatel rozhodne přerušit vývoj a vypnout server pro synchronizaci dat, uživateli stále zůstává lokální datová kopie a aplikace nadále funguje. Poté si typicky může přímo v aplikaci nastavit jinou backendovou synchronizační vrstvu – buď si ji může sám hostovat nebo využít existující.

Pro uživatele je **nižší bariéra pro vyzkoušení aplikace**. Pokud běží na platformě webu, nemusí nic explicitně stahovat a bez registrace může aplikaci využívat. Data lze vytvářet ihned, ale uživatelskou identitu k nim přiřadit až později – pokud tak uzná za vhodné po vyzkoušení aplikace a bude chtít data např. synchronizovat na jiné zařízení. Typicky je samotné použití aplikace zdarma a případně až synchronizace je placená. [15]

2.5 Důsledky pro vývojáře

Základem téměř každé aplikace v prostředí webu je sdílený stav mezi zařízeními. Vývojáři ale musí typicky většinu částí architektury implementovat pro každou aplikaci znovu – serverové API, databázi, autentizaci, autorizaci, infrastrukturu atd. To sice může být nutné pro větší systémy se specifickými požadavky, ale u menších aplikací se kód mnohdy opakuje. Pokud dokážeme synchronizační vrstvu implementovat jednou a udělat ji generickou, vývojářům výrazně **klesnou náklady na vývoj i údržbu backendové části**. Není potřeba nepřetržitě dostupný backend tým pro řešení výpadků a sníží se množství boilerplate kódu. Při tvorbě nové aplikace tedy prakticky píšeme jen frontendovou část. To také znamená, že celou aplikaci lze napsat v jednom programovacím jazyce a pro její hostování stačí pouze statický JS hosting, který bývá výrazně levnější než backendový. Kyle Simpson tento přístup nazývá „zero-server“ ve smyslu obrácení standardního mentálního modelu, kdy nejprve stavíme server s databází a API a klient je jen jeho prodloužením. U local-first přístupu místo toho již od začátku vývoje předpokládáme, že má aplikace fungovat i bez serveru.

Z pohledu vývojáře se výrazně **zjednodušuje datový tok v aplikaci**. V klient-server modelu prochází každá operace složitým řetězcem zpracování, během kterého

se několikrát převádí datový typ. Local-first tento imperativní přístup („pošli POST request, čekej na odpověď, zpracuj ji, případně zhandlej error“) nahrazuje deklarativním („zapiš do lokální DB, framework se postará o synchronizaci“). V podobném smyslu lze udělat analogii s imperativním přístupem jQuery versus deklarativní přístupem např. Reactu. U lokálního zápisu do databáze není mnoho důvodů, proč by mohl selhat – v podstatě pouze kvůli problému na straně úložiště jako plný disk nebo poškozený soubor. Pro vývojáře aplikace tak odpadá nutnost řešit chybové stavy způsobené síťovým přenosem a s nimi i nutnost používat techniku optimistic UI, která může vést k nekonzistencím v uživatelském rozhraní (operace, která byla původně označena jako úspěšná nakonec neprošla).

Další změny se týkají **monetizačního modelu a ekonomiky provozu aplikace**. U klient-server aplikací se náklady na serverovou infrastrukturu zvyšují s počtem uživatelů. U local-first modelu výpočty probíhají na zařízení uživatele, tudíž s rostoucím počtem uživatelů provozní náklady paradoxně stagnují nebo klesají [16]. Dominantní monetizační model v současném cloudovém software je SaaS s měsíčními poplatky. Provozovatel tedy prakticky prodává vlastní implementaci cloudu a je incentivizován k proprietárnímu uzamčení („zaplať, nebo ti smažeme data“). U local-first je server generický a na data nevidí, čímž se snižují možnosti na proprietární uzamčení. Lze tedy předpokládat, že monetizační model bude blíže „zaplať za to, jakou má aplikace přidanou hodnotu“. To může znamenat vývoj software na míru s jednorázovou platbou, placená synchronizace (s možností self-hostingu), nebo sponzoring open-source projektů.

2.6 Problémy a výzvy

Základní výzvou local-first přístupu je, že každé zařízení může provádět úpravy nezávisle na ostatních, čímž se z aplikace stává **distribuovaný systém** se všemi jeho specifiky. Neexistuje jeden finální stav, nýbrž všechna zařízení ke stejnému stavu pouze konvergují – tzv. „eventual consistency“. Z toho plyne také nutnost řešit konflikty, které vznikají, když dvě offline zařízení souběžně upraví stejná data. Na to se používají techniky jako CRDT a toto téma je více rozvedeno v kapitole 3.2.2. Synchronizace by měla vždy běžet v transakci, aby výpadek připojení uprostřed přenosu nezpůsobil nekonzistenci v datech.

Obecně může nastat problém u **velkých datových objemů**. Dokumentace knihovny *better-sqlite3* uvádí, že byli schopni dosáhnout rychlosti 2000 dotazů za sekundu, kde každý dotaz obsahoval joiny přes 5 tabulek a 5 až 50 KB reálných dat v celkem 60 GB databázi [17]. Pro průměrnou aplikaci je tedy lokální přístup k datům dostatečně rychlý, za předpokladu, že se dotazy optimalizují, mají správně nastavené indexy a případně aktivovaný rychlejší WAL mód (Write-Ahead Log). Limity mohou nastat u paralelního čtení mnoha MB za sekundu nebo u databází s velikostí v řádech TB. V případě větších datových objemů může být užitečné definovat prioritu synchronizace – například u chatovací aplikace se nejprve synchronizuje seznam kontaktů a až poté konkrétní zprávy. Pokud aplikace navíc běží v prohlížeči a data se ukládají skrze browser API, je nutné počítat s možností, že se prohlížeč může

rozhodnout data smazat v případě nedostatku místa na disku. Pokud je lokální persistence kritickým požadavkem, může být vhodnější místo prohlížeče použít nativní aplikaci s přímým přístupem k souborovému systému.

Jelikož je local-first aplikace distribuovaný systém, **nemůžeme dělat libovolné změny ve schématu databáze** – v síti se může nacházet aplikace s neaktualizovanou verzí, která musí být nadále schopná fungovat. Jeden z přístupů je dělat změny tzv. „append-only“, tedy sloupec lze pouze přidat, ale nelze ho odstraňovat ani přepsat existující. V případě, že při synchronizaci dat dorazí zpráva neodpovídající aktuálnímu schématu, aplikace by si ji měla uložit do separátního úložiště, tzv. karantény – a pokusit se ji zpracovat později, typicky po aktualizaci na verzi se správným schématem. Pokud je zpětně nekompatibilní změna ve schématu nezbytná, je nutno vytvořit mechanismus pro migraci dat – např. sledování verze schématu v separátní tabulce a migrační skripty na úrovni aplikace.

Pokud aplikace obsahuje šifrování, **uživatel musí spravovat svůj privátní klíč**. To může představovat výzvu, jelikož na to většina uživatelů není zvyklá. Model identity se kompletně mění z centralizovaného (e-mail a heslo na serveru poskytovatele) na decentralizovaný, kde si každý uživatel nese identitu u sebe. Již neexistuje záchranná možnost obnovy hesla. Pokud uživatel ztratí svůj privátní klíč a smaže si lokální kopii dat na všech zařízeních, o data efektivně přijde – zůstanou pouze na serveru v zašifrované podobě. Řešení existují – sociální záloha klíče (tzv. Shamir backup), fyzické zálohy nebo propojení s hardwarovým klíčem – detailněji jsou popsány v kapitole 3.3.2. Širší překážkou adopce je dominance cloudového modelu a síťový efekt – uživatelé jsou na cloud zvyklí a většinou nerozumí rozdílu mezi „data u sebe“ a „data na serveru“, ani nemají motivaci tento problém řešit. Local-first proto musí nabídnout minimálně stejně pohodlný uživatelský zážitek jako cloudové aplikace. [18]

2.7 Oblasti využití

Vhodné využití

Local-first je nejvhodnější pro aplikace, u kterých mají data lokální význam, nevyžadují okamžitou globální konzistenci a mohou být replikovány mezi zařízeními. Typicky jde o aplikace, kde veškerá data, se kterými uživatel pracuje, jsou v jeho vlastnictví. Obecně se local-first hodí pro real-time, multiplayer a offline scénáře. Právě dostupnost dat offline se projevuje jako výhoda především pokud uživatel opakovaně přistupuje ke stejným datům.

- Software pro práci se soubory (dokumenty, prezentace, grafika, audio, video)
- Software pro organizaci a produktivitu (poznámky, úkoly, kalendáře, kontakty, měření, nákupní seznam)
- Správa osobních financí a zdravotní dokumentace
- Fakturační nástroje, password managery, RSS čtečky, CRM systémy

Možné využití, složitější implementace

Některé oblasti jsou s local-first přístupem realizovatelné, ale implementačně náročnější. U sociálních sítí je potřeba překonat síťový efekt – uživatelé chtějí být tam, kde jsou ostatní. Jedním z možných přístupů u sociálních sítí může být vytváření a ukládání příspěvků primárně do lokální databáze a na jiné existující sociální platformy příspěvky pouze kopírovat. U chatovacích aplikací se musí vyřešit distribuce šifrovacích klíčů mezi uživatele. Platformy sdílené ekonomiky typu Uber nebo Airbnb závisí na centrální koordinaci nabídky a poptávky, kterou může být obtížné decentralizovat. U rozsáhlých znalostních databází typu Wikipedie působí výzvu datový objem – replikovat celou encyklopedii na každé zařízení není praktické.

- Sociální sítě
- Chatovací aplikace (textové i video)
- Platformy sdílené ekonomiky (Uber, Airbnb)
- Rozsáhlé znalostní databáze (Wikipedia)

Nevhodné využití

Naopak existují oblasti, kde local-first přístup není vhodný. Klíčovým kritériem je, zda aplikace může tolerovat eventual consistency – např. u e-shopu je potřebná okamžitá pravda o množství zboží na skladě, u bankovního převodu nelze akceptovat konfliktní stavy atd. Dále se local-first přístup nehodí pro zpracování velkých datových objemů (stovky GB až jednotky TB) ani pro náročné výpočty, které přesahují kapacitu klientského zařízení. Obecně platí, že čím více se aplikace musí spoléhat na centralizované externí zdroje z reálného světa, tím méně je local-first přístup vhodný.

- E-shopy
- Bankovní a platební služby
- Zpracování velkých souborů
- Modely strojového učení se složitými výpočty

Příklady existujících aplikací

Local-first principy jsou v různé míře využívány v již existujících aplikacích. V komerční sféře se jim přibližuje ekosystém Apple (např. Apple Notes) – fungují offline a synchronizují se přes iCloud. Problémem zde je však vendor lock-in a uzavřenost kódu. Dále lze z kategorie software pro týmovou produktivitu zmínit issue tracker Linear nebo kolaborativní workspace Colanode. Pro osobní znalostní bázi existují markdown editory Obsidian a Joplin či více strukturované Anytype a Trilium Notes. Dále existuje např. gramatický korektor Harper, editor pro typografický systém

Typst Typesetter, nástroje pro správu osobních financí Actual Budget a FinBodhi nebo RSS čtečka Alcove.

Vedle komerčních aplikací existuje také rostoucí hnutí tzv. *home-cooked software*. Robin Sloan v eseji z roku 2020 takto popsal jednoduché aplikace tvořené na míru pro úzký okruh uživatelů, např. pro rodinu [19]. Maggie Appleton tuto myšlenku dále rozvedla na Local-First Conf 2024 s pojmem *barefoot developers* v analogii s tzv. „bosými lékaři“, což byli čínští venkované se základním zdravotnickým výcvikem, kteří v 60. letech zajišťovali zdravotní péči v odlehlých oblastech [20]. Local-first se hodí právě pro tyto menší aplikace se specifickým účelem, které obzvláště dnes, v době generativních modelů, mohou tvořit i uživatelé, pro které by klasické programování bylo příliš velkou bariérou. [21]

2.8 Historický kontext

Architektura softwarových systémů se historicky vyvíjela v cyklech mezi centralizací a decentralizací výpočetních zdrojů. Každá éra přinesla nové technologie, které posunuly rovnováhu na jednu či druhou stranu – typicky v reakci na problémy éry předchozí. [22]

V počátcích výpočetní techniky v 50. a 60. letech byla veškerá logika i úložiště dat centralizována do drahých sálových počítačů (**mainframe**), které byly sdíleny mezi mnoha uživateli pomocí terminálů a dávkového zpracování – uživatel si nekupoval celý stroj, ale pouze výpočetní čas („time-sharing“). Nástup mikroprocesorů v 70. letech umožnil přesunout výpočetní výkon i úložiště přímo k uživateli ve formě **osobních počítačů** (PC). Software byl instalován i spouštěn lokálně, data se ukládala na diskety nebo pevný disk, což umožnilo vlastnictví dat, ale dané zařízení se stávalo izolovaným datovým sílem. Kolaborace mezi uživateli byla možná jedině pomocí sdílení datových nosičů s fyzickou kopií dat.

Architektura **klient-server**, která se začala prosazovat s rozvojem počítačových sítí v 80. letech a příchodem internetu v 90. letech, umožnila uživatelskou kolaboraci. Data byla uložena na jednom sdíleném serveru, ke kterému přistupovalo současně více klientských aplikací – ty sloužily jen jako rozhraní a zdrojem pravdy byl server. V roce 2000 Roy Fielding popsal principy REST (Representational State Transfer) a HATEOAS (Hypermedia as the Engine of Application State) – na základě nich poté začaly vznikat webové aplikace, které komunikují přes protokol HTTP se serverem, který vrací vyrenderované HTML stránky propojené navzájem pomocí hypertextových odkazů.

Nevýhodou tohoto přístupu, kdy je stav uložen pouze na serveru, je nutnost při každé změně překreslit celou webovou stránku. Proto se postupně začaly využívat technologie jako JSON, AJAX a WebSockets pro komunikaci se serverem bez nutnosti znovunačtení celé stránky. Zdrojem pravdy stále zůstal server, který už ovšem neposílá pouze vyrenderované HTML, ale spíše poskytuje API pro operace s daty. Klient si navíc uchovává vlastní stav a spouští JavaScript kód. Později se začaly využívat různé JavaScript frameworky, které pomocí technik jako virtual DOM, two-way data binding (MVVM) nebo Signals usnadňovaly práci s reaktivními daty.

S nástupem chytrých telefonů a tabletů mezi přelomem tisíciletí a rokem 2010 se začaly více prosazovat tzv. **cloudové aplikace**. Princip klient-server zůstal zachován, ale infrastruktura se posunula od vlastních serverů spíše k pronajímání výpočetních zdrojů třetích stran. Jedním z průkopníků tohoto přístupu byl Amazon, který v roce 2006 spustil Amazon Web Services. Dále ho následovaly služby Microsoft Azure (2010) a Google Cloud Platform (2011). Pro monetizaci aplikací se začal rozmáhat model SaaS (Software as a Service) – software přestává být *produktem*, který si uživatel koupí a vlastní ho, a stal se spíše *službou*, za jejíž používání se platí průběžně. Vznikla také poptávka po synchronizaci dat mezi více zařízení, což dalo vzniknout službám jako Dropbox, Google Drive nebo iCloud. Všechny tyto cloudové služby ovšem vedly k větší centralizaci dat na serverech třetích stran, což byl jeden z hlavních impulsů pro vznik local-first přístupu. [23]

Mnoho z principů local-first přístupu se objevovalo v praxi dlouho před tím, než byl tento pojem vymezen. Příkladem může být platforma **Lotus Notes** (později přejmenována na HCL Notes), která již v roce 1989 umožňovala replikovat dokumentové databáze mezi více zařízení v šifrované podobě. Stále sice využívala proprietární centrální server pro replikaci dat a možnosti řešení konfliktů ještě nebyly příliš rozvinuté, jednalo se ovšem o první komerčně rozšířený software, který využíval asymetrickou kryptografii pro autentizaci uživatelů [24]. Podobné principy následně prosazoval také Microsoft kolem roku 2003 s architekturou nazývanou Smart Client nebo databázový systém CouchDB, který byl vytvořen v roce 2005 jedním z bývalých vývojářů Lotus Notes. V roce 2016 poté Tim Berners-Lee představil platformu **Solid**, kde si uživatelé ukládají svá data do osobních datových úložišť nazývaných Pods a sami rozhodují, které aplikace k nim mohou přistupovat. Platforma se sice nikdy nedočkala masového rozšíření, svými principy se ale velmi přibližuje local-first přístupu. [25] [26] [27]

2.9 Budoucí vývoj

Aktuálně je local-first ekosystém velmi fragmentovaný – existuje mnoho různých knihoven s rozdílnými implementacemi synchronizační vrstvy. Martin Kleppmann v tomto kontextu argumentuje, že local-first potřebuje obecný synchronizační protokol – infrastrukturní vrstvu srovnatelnou svou rolí s DNS, na které se shodnou autoři různých knihoven [28]. Takový protokol by umožnil zaměňovat sync enginey mezi aplikacemi, čímž by se možnost proprietárního uzamčení potlačila ještě více. Uživatel by tak svá data mohl libovolně migrovat k jinému poskytovateli – což by teoreticky mohla být i některá z nynějších cloudových služeb (např. Dropbox) upravena pro účely local-first.

Adopci local-first přístupu mohou urychlit vnější faktory. Podpora klíčových technologií v prohlížečích (OPFS, WebAssembly, WebAuthn) se postupně zlepšuje, ale mnoho možností stále v prohlížečích chybí. Jedno z dalších témat je regulační tlak ze strany Evropské unie. GDPR zakotvuje právo na přenositelnost dat, které ukládá správcům poskytnout uživateli jeho data ve strukturovaném a strojově čitelném formátu. Digital Markets Act jde ještě dál a vyžaduje nepřetržitý přístup

k datům v reálném čase a jejich přenositelnost. Local-first tyto regulatorní požadavky splňuje nativně – data jsou už z principu u uživatele v otevřeném formátu. Dalším významným faktorem je rozmach umělé inteligence. S rozšiřováním lokálních modelů (Llama, Gemini Nano) roste význam lokálně dostupného kontextu – uživatel může mít personalizovaného asistenta pracujícího nad jeho daty bez nutnosti posílat citlivé informace do cloudu. Local-first přístup se proto přirozeně hodí pro budování lokální knowledge base, nad kterou lze provádět techniky jako RAG (Retrieval-Augmented Generation).

V ještě dlouhodobějším horizontu se otevírá otázka, zda by principy local-first nemohly být zabudovány přímo na úrovni operačního systému. Aktuálně používáme stále klasickou unixovou abstrakci z 70. let s hierarchickým souborovým systémem, kde jsou soubory pouhé sekvence bytů bez hlubšího sémantického významu. To je nedostačující pro scénáře, kde chceme umožnit dvěma uživatelům na jednom počítači editovat stejný soubor a poté změny sloučit, což vyžaduje vyšší úroveň abstrakce nad daty. Teoreticky by local-first operační systém mohl mít CRDT funkce zabudované přímo na úrovni souborů, čímž by synchronizace a bezkonfliktní slučování dat byly dostupné všem aplikacím bez nutnosti vlastní implementace [28].

Sdílení dat mezi aplikacemi

Další z možných vizí local-first přístupu je umožnit sdílení stejných dat mezi více aplikací. Tatáž data by mohla být jednou zobrazena např. jako formulář, jindy jako tabulka nebo interaktivní diagram – přičemž změny provedené v jedné aplikaci se okamžitě projeví i v ostatních. Příkladem use case může být např. v aplikaci na poznámky, kde si chceme v textu udělat odkaz na kontakt – propojí se s aplikací na kontakty a nabídne dropdown možností; samotný kontakt poté funguje jako odkaz do druhé aplikace.

Technicky tato vize naráží na problém, že web byl od svého vzniku postaven na **Same-Origin Policy**, která z bezpečnostních důvodů zabraňuje aplikacím z jiných domén přistupovat k datům. Jedinou možností pro sdílené data v prostředí webu tak je master aplikace, která koordinuje konkrétní aplikace – musí být spolu ovšem svázané a na stejné doméně. V nativním prostředí tento problém řeší Android Intents a Content Resolver – aplikace deklaruje, jaké typy dat zvládá zpracovat, a ostatní aplikace je mohou volat bez znalosti konkrétní implementace. **Web Intents** od Googlu byl pokus o zavedení stejného konceptu v prostředí webu, ovšem selhal v roce 2014 kvůli kombinaci nezájmu prohlížečových výrobců, bezpečnostních obav a absenci webového registru aplikací. Existuje také **Web Share API**, které ale poskytuje pouze jednosměrný přenos dat. Skupina Ink & Switch v této oblasti pracuje na řešení zvaném Embark, které využívá sdílené dokumenty s dynamickými pohledy.

3 Technologie a architektura

3.1 Ukládání dat

U local-first aplikace je v první řadě nutné zvolit technologii pro ukládání dat. Nejjednodušší možností je zakódovat celý stav aplikace v Base64 řetězci přímo do URL, čímž vzniká odkaz s vestavěným obsahem. Tento přístup se hodí pouze pro velmi jednoduché aplikace – například služba pro sdílení krátkých úryvků textu. Druhou možností je implementovat vlastní datovou strukturu. To sice poskytuje plnou kontrolu nad formátem dat, ale prakticky to znamená reimplementaci celého databázového systému, což je komplexní obor a mnoho práce. Třetí a pravděpodobně nejrozšířenější cestou je použít SQL databázi. Relační model, na kterém SQL stojí, je prověřen mnoha lety používání a ukazuje se jako vhodná univerzální volba pro většinu aplikací [29].

Nativní aplikace – React Native, Expo, Flutter, Android, iOS, ale i hybridní Electron a Capacitor – mají oproti webovým aplikacím jednu klíčovou výhodu v kontextu datového úložiště: přímý přístup k souborovému systému bez zprostředkování prohlížečem. To snižuje riziko neočekávaného smazání dat a odpadají kvóty na maximální velikost úložiště, které může stanovit prohlížeč. Na druhou stranu je třeba zajistit bindingy pro daný jazyk a platformu, které jinak prohlížeč poskytuje automaticky. Tato kapitola se dále zaměřuje primárně na webovou platformu.

3.1.1 SQLite

SQLite je embedded relační databáze, která je celá uložena v rámci jednoho souboru na disku. Na rozdíl od klasických databází jako MySQL nebo PostgreSQL nevyžaduje SQLite k provozu samostatný server, ale je přímo integrován do aplikace jako knihovna. Je napsán v jazyce C a jeho zdrojový kód je distribuován v samostatném souboru, který obsahuje kompletní databázový engine. Hlavním důvodem spolehlivosti SQLite je rozsáhlé množství testů: zdrojový kód obsahuje přibližně $590 \times$ více řádků testů než samotné logiky (cca 92 milionů versus 155 tisíc) [30]. Bindingy existují pro téměř každý programovací jazyk a jedná se o jeden z nejvícekrát nasazených softwarových produktů na světě – vedle knihoven jako zlib, libpng a libjpeg [31]. SQLite lze najít v zařízeních s Androidem, iOS a Windows, v prohlížečích, v jazycích PHP a Python, ale i v televizích, automobilových multimediálních systémech nebo například v letadlech Airbus.

Přestože může být SQLite vnímán jako jednoduchá databáze, nabízí řadu pokročilých funkcí, které nemusí být na první pohled zřejmé. Implementuje většinu standardu SQL-92 a také mnoho prvků z novějších verzí tohoto standardu – včetně poddotazů, pohledů, triggerů a CTE (Common Table Expressions). Od verze 3.38.0 obsahuje SQLite nativní podporu pro práci s JSON daty, což umožňuje kombinovat relační a dokumentový přístup v rámci jedné databáze. Další užitečnou vlastností jsou tzv. *generated columns*, které poskytují možnost automaticky odvozovat hodnotu sloupce dle jiných sloupců. Pro full-text vyhledávání je k dispozici rozšíření FTS5, které poskytuje tokenizaci, řazení výsledků dle relevance a pokročilé operátory. Z hlediska typového systému je SQLite unikátní svým dynamickým typováním – deklarace typu ve schématu je spíše doporučením a buňka může obsahovat hodnotu libovolného typu. Pro aplikace, které vyžadují přísnější kontrolu typů, nabízí SQLite od verze 3.37.0 také striktní typovací režim [32]. SQLite lze šifrovat buď pomocí oficiálního rozšíření SQLite Encryption Extension nebo komunitního řešení SQLite3 Multiple Ciphers pomocí direktiv `PRAGMA cipher` a `PRAGMA key`.

Některé z vlastností popsaných standardem SQL-92 se v SQLite nenachází. Omezená je možnost úpravy schématu (`ALTER TABLE`) – sloupce lze přidávat, mazat a přejmenovat, ale nelze upravit jejich datový typ a definované omezení (`constraint`). U triggerů lze využít pouze režim `FOR EACH ROW` a chybí podpora pro režim `FOR EACH STATEMENT`. Pohledy (`views`) jsou v SQLite dostupné pouze pro čtení, což lze obejít vytvořením `INSTEAD OF` triggeru, který reaguje na pokus o zápis do pohledu. SQLite také neimplementuje příkazy `GRANT` a `REVOKE` pro řízení přístupu, jelikož nemá žádný koncept databázových uživatelů, hesel ani rolí.

Další z možných limitů SQLite je souběžný přístup. V základním režimu (`rollback journal`) drží databáze při zápisu exkluzivní zámek, tedy nikdo jiný nemůže číst ani zapisovat. V roce 2010 byl přidán režim `WAL` (`Write-Ahead Logging`), který data průběžně zapisuje do speciálního `WAL` souboru a do hlavní databáze se přelévají až hromadně na pozadí během operace zvané `checkpointing`, díky čemuž zápisy neblokují čtení. Nejnovější režim se označuje `WAL2` a jedná se o experimentální koncept, který se zatím nachází v oddělené vývojové větvi. Přidává druhý `WAL` soubor a tím řeší problém zablokování zápisů při provádění `checkpointingu` – nyní lze zápisy přesměrovat do druhého souboru než se první znovu uvolní.

Možnými alternativami k SQLite v kontextu `local-first` jsou **PGLite** – PostgreSQL zkompileované do `WebAssembly` – a **Turso**, `in-process SQL` databáze napsaná v Rustu postavená na forku SQLite zvaném `libSQL`. PGLite se může hodit v případech, kdy aplikace vyžaduje pokročilé funkce PostgreSQL, jako je např. typ `JSONB` s efektivním binárním ukládáním nebo práce s vektory s rozšířením `pgvector`. Turso přidává podporu souběžných zápisů, které SQLite v základní verzi nepodporuje. V době psaní této práce je ovšem Turso stále ve vývoji a není doporučeno pro produkční použití [33].

3.1.2 Vývoj webových technologií

Možnosti ukládání dat na straně klienta byly po dlouhou dobu velmi omezené. Nejstarším dostupným mechanismem jsou `HTTP cookies`, zavedené v polovině 90. let,

které sloužily primárně k uchování malého objemu stavových informací (typicky do 4KB). Standardizovaný přístup pro ukládání aplikačních dat přinesla až specifikace Web Storage (`localStorage` a `sessionStorage`), která se stala součástí HTML5 a nabídla synchronní key-value úložiště s kapacitou řádově 5–10MB. Ani Web Storage však nebyl navržen pro práci s většími či strukturovanými daty a na klientu stále chyběla plnohodnotná databáze. Jak shrnují autoři eseje Ink & Switch: *„Přestože bylo vynaloženo velké úsilí, aby byly webové prohlížeče přátelštější k offline použití (manifesty, localStorage, service workery a Progressive Web Apps), webová architektura zůstává fundamentálně serverově orientovaná a offline podpora se ve většině aplikací řeší až zpětně.”* [1] [34]

Prvním pokusem o překonání této limitace bylo **Web SQL** – rozhraní navržené v roce 2009, které zpřístupňovalo SQLite databázi přímo z prostředí prohlížeče. Iniciativa se však neprosadila z několika důvodů: standard nebyl nikdy formálně specifikován, závisel na konkrétní verzi SQLite (3.6.19), API bylo nevhodně navrženo a Firefox jej kompletně odmítl implementovat. W3C proto Web SQL v roce 2010 označilo jako deprecated. [35]

Jako náhrada za Web SQL byla v roce 2011 uvedena **IndexedDB** – asynchronní objektová databáze v prohlížeči. Na rozdíl od Web Storage umožňuje IndexedDB ukládání strukturovaných dat včetně binárních objektů (Blob), podporuje indexy, transakce a nemá pevně daný limit kapacity (závisí na procentu volného místa na disku). Nejedná se však o relační databázi: IndexedDB neposkytuje dotazovací jazyk, data se prohledávají pomocí kurzorů a indexů. Její API také obsahuje určité limitace, jako např. nemožnost přidávat nové *object stores* za běhu (lze pouze při otevírání databáze, což vynucuje odpojení ostatních tabů) a mezi prohlížeči se může rychlost IndexedDB lišit v důsledku odlišné implementace [36]. IndexedDB se přesto stala na řadu let jedinou praktickou možností pro ukládání větších objemů dat ve webových aplikacích.

Vývojáři proto hledali cestu, jak na webu provozovat plnohodnotnou relační databázi. Projekt **sql.js**, existující již od roku 2014, kompiloval jádro SQLite do WebAssembly. Zásadní limitací ale byla pouze in-memory funkčnost – databázi šlo importovat a exportovat jako celek, nikoli persistentně ukládat jednotlivé změny na disk [37]. V roce 2021 zkoumal James Long v rámci knihovny **absurd-sql** zajímavý přístup, který implementoval SQLite nad IndexedDB: místo načítání celé databáze do paměti ukládal samotné SQLite bloky (stránky o velikosti 4KB) přímo do IndexedDB, čímž v podstatě emuloval souborový systém. Paradoxem bylo, že SQLite takto provozované nad IndexedDB překonávalo v benchmarkách samotnou IndexedDB – především díky dávkování I/O operací a cachováním jednotlivých stránek. Výkon byl stále přibližně 50–100× pomalejší než nativní SQLite na disku, ale pro řadu use cases již plně dostačující [36].

Přelomovou novinkou byla kombinace **SQLite zkompilevaného do WebAssembly** společně s rozhraním **Origin Private File System** (OPFS). Koncem roku 2022 vydali vývojáři SQLite oficiální WASM build [38] a OPFS začalo být dostupné ve všech hlavních prohlížečích počátkem roku 2023. Na rozdíl od absurd-sql, které muselo emulovat souborový systém nad IndexedDB, může SQLite díky

OPFS backendu pracovat přímo se soubory na disku, čímž odpadá režie spojená s IndexedDB. [39]

Tabulka 3.1: Porovnání technologií pro persistenci dat v prohlížeči

Technologie	Limit kapacity	Typ dat	API model
Cookies	4 KB	string (key-value)	sync
Local/Session Storage	5–10 MB	string (key-value)	sync
IndexedDB	řádově GB*	strukturovaná data	async
OPFS	řádově GB*	binární soubory	sync/async**

* Závisí na množství volného místa na disku.

** Synchronní přístup dostupný pouze z Web Workeru.

3.1.3 Origin Private File System

Origin Private File System (OPFS) je virtuální souborový systém přístupný z prohlížeče, který se váže k originu stránky. Má stromovou strukturu a poskytuje standardní operace se soubory a adresáři. Na rozdíl od starších pokusů o přístup k souborům z webu (File System Access API), které se neprosadily kvůli obavám o bezpečnost, OPFS díky svému sandboxovému přístupu nevyžaduje explicitní potvrzení od uživatele pro souborovou manipulaci. Soubory uložené v OPFS nejsou na disku reprezentovány 1:1 – prohlížeč jejich umístění obfuskuje. Jejich obsah je ovšem nezměněný, tudíž existuje možnost je na disku dohledat. Tento problém je podrobněji rozebrán v kapitole 3.5.

OPFS podléhá stejným kvótám o velikosti úložiště jako IndexedDB a Cache API, prohlížeč se tedy může kdykoliv rozhodnout data v OPFS smazat. Pravidla se u jednotlivých prohlížečů liší, typicky se ovšem jedná o překročení určitého procenta celkového volného místa na disku, u Safari ještě navíc o nenavštívení dané stránky po dobu 7 dní [40]. Aplikace si navíc může zažádat o povolení tzv. persistentního režimu (zavoláním metody `navigator.storage.persist()`), který by měl data v OPFS chránit před automatickým smazáním. Vyhodnocení, zda bude persistentní režim povolen, se opět liší napříč prohlížeči – Chrome a Safari používají heuristiku dle uživatelské interakce s danou stránkou a Firefox vyžaduje od uživatele explicitní potvrzení. [41] [42]

Ze všech dostupných možností v prohlížeči je OPFS nejbližší nativnímu souborovému přístupu. Soubory lze číst a zapisovat byte po byte a je podporován náhodný přístup k libovolné pozici v souboru (random access writes). Právě díky důrazu na výkonnost je vhodnou volbou pro použití společně s SQLite. OPFS nabízí dvě varianty API. Při přístupu z hlavního vlákna je k dispozici pouze asynchronní API, aby se předešlo zamrznutí uživatelského rozhraní. Jen přístup z Web Workeru umožňuje použít synchronní API, které je jednodušší na implementaci i ladění a také umožňuje dosáhnout vyššího výkonu – odpadá veškerá režie s Promise/Async komunikací. Doporučenou praxí je proto vždy přistupovat k OPFS z Web Workeru. [43]

3.1.4 Web Workers

JavaScript v prohlížeči běží v jednom vlákně. Každý výpočetně náročný úkol tak blokuje hlavní vlákno a způsobuje zamrznutí uživatelského rozhraní. **Web Workery** umožňují spouštět skripty v samostatném vlákně operačního systému. Worker nemá přístup k DOM (včetně objektu `window` a `document`) ani k Web Storage API, ale má přístup k síťové komunikaci (WebSockets, `fetch`), časovačům (`setTimeout()`, `setInterval()`) a datovým úložištím (OPFS a IndexedDB). Web Workery mají nezanedbatelnou režii při spuštění a paměťovou spotřebu, nejsou proto vhodné k masivnímu paralelismu, ale spíše pro dlouhodobě běžící úlohy. Typickým příkladem použití mohou být operace jako šifrování, komprese, parsování a pro local-first aplikace především přístup k databázi uložené v OPFS skrze synchronní API.

Web Workery se dělí na tři typy. Základní **Dedicated Worker** je svázan se skriptem, který ho vytvořil, a komunikuje výhradně s ním. Zavřením dané stránky také automaticky končí životní cyklus workeru. **Shared Worker** je sdílený mezi více záložkami se stejným originem, což je užitečná vlastnost pro zajištění, že více otevřených záložek jedné aplikace používá stejnou instanci databáze nebo WebSocket spojení. **Service Worker** se od těchto dvou odlišuje – běží na pozadí nezávisle na otevřených tabech, může zachytávat a modifikovat síťové požadavky, umožňuje cachování a tvoří základ pro PWA a push notifikace.

Nejjednodušší způsob komunikace mezi hlavním vláknem a Workerem je voláním metody `postMessage()`. Předávaná data nejsou sdílena, nýbrž kopírována skrze tzv. Structured Clone Algorithm, který vytvoří hlubokou kopii daného objektu. Při přenosu velkých datových objemů, kdy režie kopírování představuje výkonnostní problém, lze využít Transferable objekty, které místo kopírování dat pouze přesouvají ukazatel na paměť. Třetí možností komunikace je `SharedArrayBuffer`, který reprezentuje blok sdílené paměti, ke které může současně přistupovat jak hlavní vlákno, tak worker. Aby se zamezilo souběhu nebo uváznutí při přístupu k datům, využívá se objekt `Atomics`, který poskytuje atomické operace nad `SharedArrayBuffer` a umožňuje vláknům synchronně čekat (`Atomics.wait()`), než je znovu probuzeno z hlavního vlákna (`Atomics.notify()`). `SharedArrayBuffer` vyžaduje z důvodu bezpečnosti HTTPS kontext a dvě nastavené HTTP hlavičky pro vynucení cross-origin izolace. Důvody těchto bezpečnostních omezení jsou popsány v kapitole 3.5.1.

3.2 Synchronizace dat

Synchronizace dat v local-first je proces, kterým se více nezávislých replik téhož datového setu dostává do konzistentního stavu. Pro transport dat lze využít libovolný komunikační kanál: WebSocket, HTTP, Bluetooth, Wi-Fi Direct, NFC a teoreticky také QR kód nebo zvuk. Transportní vrstva je v local-first architektuře záměrně oddělena od aplikační logiky. Díky tomu lze využít i existující cloudové služby jakožto backend pro synchronizaci, aniž by aplikace byla na dané službě závislá [1].

Při synchronizaci musíme řešit dva základní problémy: konflikty vzniklé souběžnými úpravami (jeden uživatel přesune položku a druhý ji mezitím smaže) a nespolehlivé pořadí příchozích změn (mohou na každé zařízení přijít v jiném pořadí,

potřebujeme je kauzálně seřadit). Oba problémy jsou rozvedeny v následujících podkapitolách. Otázkou také je, kdy synchronizaci provést – zda při každé změně, v pravidelných intervalech nebo např. při zavření aplikace.

Skutečné mazání dat (hard delete) v distribuovaném prostředí komplikuje synchronizaci. Pokud je záznam na jednom zařízení smazán, jiné zařízení při synchronizaci nemá šanci poznat, zda byl odstraněn, nebo ho první zařízení pouze zatím nemá synchronizovaný. Standardním vzorem je proto **soft delete** – záznam se místo smazání pouze označí příznakem **isDeleted**, díky tomu se operace smazání doručí všem klientům jako ostatní změny. Vedlejším efektem je snadná implementace funkce koš nebo undo (time travel). Nevýhodou je přenos většího množství dat kvůli nutnosti udržovat již smazané záznamy – to lze případně řešit vlastním garbage collectorem, který data po nějaké době skutečně smaže.

3.2.1 Distribuované systémy

Distribuovaný systém lze definovat jako skupinu uzlů (zařízení), kde si každý udržuje vlastní repliku dat a může s nimi pracovat nezávisle na ostatních – s tím, že jednou za čas sdílí provedené změny s ostatními. Uzly mezi sebou nesdílejí paměť a komunikují spolu výhradně posíláním zpráv přes síť. Local-first je tedy ze své podstaty distribuovaný systém [44].

Rámcem pro uvažování o distribuovaných systémech je CAP theorem formulovaný Ericem Brewerem. Popisuje, že žádný distribuovaný systém nemůže současně garantovat všechny tři vlastnosti: **Consistency** (každé čtení vrací poslední zapsanou hodnotu), **Availability** (každý požadavek dostane odpověď) a **Partition tolerance** (systém funguje i při výpadku komunikace mezi uzly). Protože výpadkům sítě se v reálném světě nelze vyhnout (P je tedy dané), volba se prakticky redukuje na kompromis mezi konzistencí a dostupností. Zatímco tradiční serverové databáze (např. PostgreSQL) volí konzistenci a při ztrátě spojení s primárním uzlem raději požadavek odmítnou, local-first architektura upřednostňuje dostupnost – každé zařízení musí fungovat i plně offline. Plná konzistence je tedy potlačena na tzv. **eventual consistency** a jedná se o AP systém.

Konkrétním příkladem decentralizovaného protokolu je **Nostr** (Notes and Other Stuff Transmitted by Relays), který slouží pro distribuci podepsaných JSON objektů přes WebSockets [45]. Architektonicky sdílí řadu principů s local-first: identita uživatele je postavena na kryptografii a servery se označují jako Relays a nejsou autoritativní. Protokol definuje standardy pomocí NIPs (Nostr Implementation Possibilities).

Peer-to-peer

Otázkou zůstává, zda komunikovat čistě peer-to-peer (P2P) nebo využívat Relay server. P2P se vyplácí ve dvou scénářích: u statických dat, kde lze zátěž distribuovat mezi klienty (BitTorrent, linuxové distribuce), a v prostředích s plnou kontrolou nad sítí (lokální síť v letadle nebo smart home systém). V otevřeném internetu ovšem P2P naráží na problém, že většina koncových zařízení je schovaná za NAT

(typicky domácím routerem), tudíž mají pouze lokální adresu a ostatní zařízení se na ně nemohou přímo připojit. Pro překonání NAT existují techniky jako *hole punching*, jejich úspěšnost ovšem není zaručena a v určitých konfiguracích sítě (zejména symetrický NAT nebo příliš restriktivní firewall) selhávají. [10]

V kontextu webu ještě navíc prohlížeče nemají přístup k nízkoúrovňovým síťovým API, které jsou potřebné pro standardní P2P protokoly (nelze otevřít TCP socket ani přímo naslouchat na portu). Jedinou existující možností je protokol **WebRTC**, který umožňuje otevřít přímou P2P komunikaci mezi dvěma prohlížeči v reálném čase. Ani ten ovšem není čistě P2P – zaprvé spoléhá na tzv. signaling server pro navázání prvotního spojení a výměnu metadat, zadruhé v případě, že přímé připojení selže (typicky kvůli NAT), je celá komunikace přesměrována přes tzv. TURN server.

V praxi tedy čistě P2P přístup často naráží na síťové limity a využití serveru se jeví jako lepší možnost. Local-first z toho důvodu servery nezavrhuje, pouze se je snaží držet co nejvíce generické, zaměnitelné a nezáviset na nich. [46]

3.2.2 Conflict-free Replicated Data Types

Conflict-free Replicated Data Types (CRDT) slouží k řešení konfliktů vzniklých souběžnými úpravami. Jedná se o datové struktury, které dokáží deterministicky sloučit změny přicházející v libovolném pořadí tak, aby byl výsledný stav na všech zařízeních totožný. Operace s CRDT tedy musí být komutativní a idempotentní. Každý uživatel může aktualizovat vlastní kopii nezávisle na ostatních, repliky mohou být dočasně rozdílné, ovšem po propagaci všech změn konvergují ke stejnému stavu. Termín CRDT byl poprvé použit v roce 2011 [47].

Dříve než existovaly CRDT, technika využívaná pro kolaborativní editaci byla **Operational Transformation** (OT). Existuje od roku 1989 a byla implementována např. v Google Docs nebo Microsoft Office Online. OT ovšem vyžaduje centrální server, který ukládá globální pořadí operací a transformuje příchozí operace dle toho, co protistrana ještě neviděla – proto se nehodí pro local-first.

CRDT lze rozdělit do tří kategorií dle způsobu propagace změn. **State-based CRDT** posílají při synchronizaci vždy celý stav objektu a příjemce ho sloučí se svým lokálním stavem skrze definovanou *merge* funkci. Výhodou je jednoduchost implementace a robustnost vůči výpadkům sítě, nevýhodou je vysoký objem přenášených dat. **Operation-based CRDT** naopak posílají pouze jednotlivé operace, ale vyžadují spolehlivý mechanismus pro kauzální uspořádání operací. Množství přenesených dat je díky tomu nižší. **Delta-state CRDT** jsou kompromisem – fungují jako state-based, ale namísto celého stavu posílají pouze deltu od posledního synchronizovaného bodu.

Dále u CRDT existují tyto základní typy datových struktur, ze kterých lze poté skládat složitější struktury (mapy a listy):

- **G-Counter** (Grow-only Counter) – pouze inkrementální čítač
- **PN-Counter** (Positive-Negative Counter) – kombinace dvou G-Counterů, umožňuje inkrement i dekrement

- **OR-Set** (Observed-Remove Set) – množina sledující individuální přidání a odebrání prvku
- **LWW-Register** (Last Write Wins Register) – uchovává nejnovější hodnotu dle časové značky
- **MV-Register** (Multi Value Register) – uchovává všechny konfliktní hodnoty, řešení nechává na aplikaci

Nejjednodušší prakticky použitelnou strategií je **Last Write Wins**, která se hodí hlavně pro atomická data jako číselná hodnota, enum nebo checkbox. Typicky se implementuje na úrovni sloupce v databázi, konflikt tedy vznikne pouze při souběžné změně stejného pole u stejného objektu. LWW přestává být dostačující u delších textových dat (především rich-text). Pokud dva uživatelé upraví stejný odstavec, výsledkem by bylo buď úplné zahození jedné verze nebo nesmyslný text s prokládanými znaky. Pro tyto případy existují specializované textové CRDT se složitější implementací. Je ovšem nutné si uvědomit, že CRDT zaručují pouze konvergenci stavu, nikoliv zachování záměru uživatele – pokud např. jeden uživatel navýší u receptu množství cukru a druhý mezitím změni cukr na med, algoritmus nedokáže automaticky rozhodnout o správném výsledku a tato volba musí být ponechána na aplikaci/uživateli. Ve stejném smyslu CRDT nedokáže řešit aplikační invarianty – např. pravidlo, že stav účtu nesmí klesnout pod nulu.

Ink & Switch ovšem poznamenává, že se konflikty v praxi neobjevují tak často, jak by se mohlo zdát. Jedním důvodem pro to je granulární rozdělení CRDT na úrovni databázových sloupců (na rozdíl od např. Gitu, kde stačí upravit stejný řádek souboru), druhým důvodem je intuitivní domluva mezi uživateli ohledně rozdělení práce takovým způsobem, aby si do ní navzájem co nejméně zasahovali. V řešení stále zůstávají některé problémové vzorce [48]: již zmíněné prokládání znaků v textu, přesouvání položek v listech, přesouvání podstromů ve stromových strukturách a v neposlední řadě rostoucí datový objem kvůli uchovávání kompletní historie operací. Pro představu o velikostech: Martin Kleppmann zaznamenal při psaní akademické práce přibližně 360 000 CRDT zpráv (na úrovni stisku jednotlivých kláves) o surové velikosti zhruba 36 MB, které se pomocí komprese implementované v knihovně Automerge (tree pruning) podařilo zmenšit na přibližně 300 KB. [49] [50] [51]

3.2.3 Hybrid Logical Clocks

Hybrid Logical Clocks (HLC) slouží k řešení kauzálního uspořádání operací. Jejich pořadí je klíčové zejména pro Last Write Wins strategii. Naivním řešením by bylo použít hodiny zabudované v zařízení (tzv. *wall clock*) jako časovou značku. To ovšem naráží na problém nespolehlivosti – hodiny mohou být pozpátku (z důvodu zaseknutí) nebo uživatelem ručně přepsány na libovolnou hodnotu. Pokud by je uživatel posunul např. o 24h dopředu a provedl změny, jiné zařízení se správným časem by 24h nemohlo vygenerovat časovou značku, která by byla vyhodnocena jako

pozdější a jeho změny by byly ignorovány. Druhým problémem by byla nemožnost odlišit dvě události v rámci stejné milisekundy.

HLC toto řeší kombinací fyzického času a logického čítače. Tím se zaručuje, že budou zaprvé všechny zprávy vytvořené v rámci jednoho zařízení správně seřazeny a zadruhé, že po přijetí zprávy z jiného uzlu budou mít všechny nově vytvořené zprávy novější timestamp než zpráva přijatá. Oproti Lamport Clocks, které produkují pouze pořadové číslo bez vazby na reálný čas, jsou HLC navíc lidsky čitelné.

Jejich implementace obsahuje tedy tři složky: **wall clock timestamp**, **logický čítač** a **unikátní identifikátor daného uzlu**. Prakticky fungují tak, že se při vytvoření nové zprávy porovná aktuální wall clock s posledním uloženým HLC. Je-li wall clock novější, použije se a logický čítač se resetuje na nulu. Pokud je wall clock pozadu, použije se uložené HLC a čítač se inkrementuje pro odlišení záznamů. Při přijetí nové zprávy se HLC aktualizuje na maximální hodnotu z vlastního a přijatého stavu. Unikátní identifikátor zařízení slouží jako poslední možnost pro odlišení záznamů s totožným timestampem i čítačem. HLC lze serializovat do textového řetězce a jejich porovnání tak probíhá lexikograficky, postupně přes jednotlivé složky. Ochranou proti případům, kdy si uživatel posune čas dopředu, je na úrovni synchronizace odmítat zprávy, jejichž *timestamp* se výrazně odchyluje od aktuálního času přijímacího zařízení – tento limit se obvykle označuje jako *max drift* a v praxi se nastavuje na jednotky minut. [52] [53]

3.2.4 Range Based Set Reconciliation

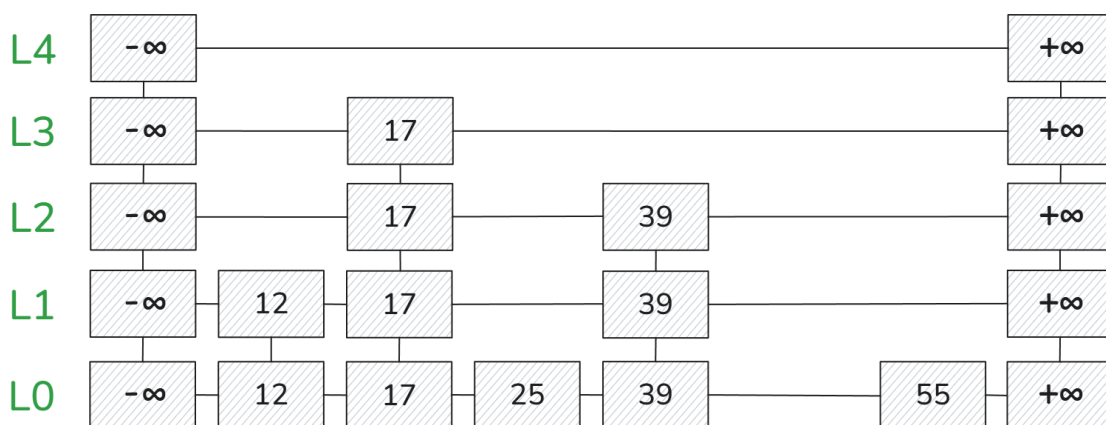
Synchronizace dat vyžaduje, aby si obě strany porovnaly obsah svých databází a vyměnily si pouze chybějící záznamy (nechceme pokaždé posílat všechny). **Range-Based Set Reconciliation** (RBSR) je divide-and-conquer algoritmus navržený přesně pro tento problém [54]. Funguje tak, že se nejprve obě množiny seřadí dle stejného klíče. Poté se porovnávají hashe daných rozsahů – pokud se hash na obou stranách shoduje, celý rozsah je identický a může být přeskočen. Pokud se hashe liší, rozsah se rekurzivně rozdělí na podrozsahy a proces se na každém z nich opakuje, dokud nejsou identifikovány konkrétní rozdíly. Až poté se přenášejí samotná data rozdílných záznamů. Jednotlivé větve rekurze jsou na sobě nezávislé, algoritmus je tedy přirozeně vhodný k paralelizaci.

Klíčovým parametrem je **branching factor**, tedy na kolik podrozsahů se každý rozsah dělí. Typicky se volí mocniny dvou – vyšší faktor znamená menší hloubku rekurze a méně roundtripů, ale větší objem dat přenesených v každém kroku. Při branching factoru 16 vychází díky logaritmické posloupnosti synchronizace velmi efektivně: pro milion záznamů jsou potřeba max. tři roundtripy, pro miliardu záznamů max. čtyři. Právě síťový roundtrip představuje největší zpomalení synchronizace a díky jejich minimalizaci lze efektivně škálovat. RBSR není jediným algoritmem pro tento účel – alternativou je např. RIBLT (Rateless Invertible Bloom Lookup Tables), který využívá Automerge.

Skip List

RBSR pro své fungování potřebuje sub-algoritmus, který dokáže rychle spočítat hash libovolné výšece dat v seřazené množině. Pro tento účel se používá technika **inkrementálního hashování**, která potřebuje datovou strukturu binárního stromu [55]. Nevýhodou je, že binární strom vyžaduje explicitní vyvažovací operace při každém vložení a smazání, což vede ke komplexnějšímu kódu i horšímu výkonu z důvodu režie.

Možnou alternativou k binárnímu stromu je **Skip List**, probabilistická datová struktura navržená Williamem Pughem. Jedná se o vícevrstvý spojový seznam, kde nejnižší vrstva obsahuje všechny prvky v seřazeném pořadí a vyšší vrstvy obsahují různé podmnožiny prvků a slouží jako zkratky při vyhledávání. Při vkládání se nový prvek zařadí náhodným losováním do několika vrstev. S parametrem pravděpodobnosti přidání do další vrstvy $p = 1/2$ mají všechny operace (vyhledání, vložení, smazání) průměrnou složitost $\mathcal{O}(\log n)$. Skip List tedy zachovává stejné vlastnosti jako vyvážený binární strom bez nutnosti implementovat vyvažování. Cenou za to je vyšší paměťová náročnost a také, že logaritmická složitost platí pouze průměrně, nikoliv v nejhorším případě. Pro většinu praktických použití je to ovšem dostačující.



Obrázek 3.1: Datová struktura Skip List

3.3 Šifrování dat a správa klíčů

Díky generické architektuře Relay serveru se přímo nabízí šifrovat všechna data jakožto výchozí stav. To typicky provádíme na dvou místech. Prvním je šifrování „at-rest” – dat uložených v lokální databázi na disku zařízení, čímž k nim zamezíme útočníkovi přístup v případě kompromitace zařízení. Druhým místem je šifrování „in-transit” – při synchronizaci mezi zařízeními, čímž zamezíme útočníkovi přístup k datům v případě kompromitace Relay serveru.

Základní rozdělení šifrování je dle použitého typu šifry na symetrické a asymetrické. Obě uvedená typická místa šifrování v local-first využívají šifru symetrickou, hlavně z důvodu výpočetní rychlosti a kvantové odolnosti. Asymetrické

šifrování se typicky používá pouze pro bezpečnou výměnu symetrického klíče skrze nezabezpečený kanál, což je popsáno v kapitole 3.4.1.

3.3.1 Historie šifrování

V současné době je využívání silných šifrovacích algoritmů bráno jako standard. Jak píše kryptograf Daniel J. Bernstein: „Pro většinu kryptografických operací existují všeobecně uznávaná primitiva, která byla důkladně prozkoumána, a jejich prolomení je na jakémkoli stávajícím clusteru považováno za výpočetně neproveditelné” [56]. To ovšem nebylo vždy samozřejmostí a jedná se o výsledek dlouhého politického boje, který se nazývá **Crypto Wars**. Vláda Spojených států dříve označovala kryptografický software jako munici, která podléhala stejným exportním omezením jako tanky, rakety či bojová letadla. Jejich export mimo území USA byl bez vládní licence považován za federální zločin. Softwarové firmy proto musely pro mezinárodní trh dodávat záměrně oslabené verze produktů. Příkladem je prohlížeč Netscape Navigator, který existoval ve dvou variantách – pro domácí trh s 128bitovým SSL a pro export s 40bitovým SSL, které je kryptograficky triviální na prolomení.

Důležitým momentem byl rok 1991, kdy Phil Zimmermann vydal software **Pretty Good Privacy** (PGP) pro silné asymetrické šifrování e-mailů dostupný zdarma jako open-source s cílem, aby šifrování mohl používat kdokoli. Americký celní úřad zahájil tříleté vyšetřování pro podezření z nelegálního exportu munice, které nakonec sice skončilo bez obvinění, ale stalo se symbolem konfliktu mezi státním dohledem a právem na soukromí. Zimmermannovi podporovatelé začali tisknout zdrojový kód PGP jako knihu a legálně ji exportovat mimo USA – tištěné materiály totiž podléhaly ochraně prvního dodatku americké ústavy. Jednou z motivací pro vytvoření PGP byl také návrh Senátu USA (Senate Bill 266), který nařizoval výrobcům komunikačních zařízení zabudování zadních vrátek, které by vládě umožnily prolomit šifrování [57].

Tato zadní vrátka byla známá jako Clipper Chip. Šlo o hardwarový šifrovací modul navržený NSA v roce 1993, který sice používal silné šifrování, ale kopie každého šifrovacího klíče (tzv. *key escrow*) by byla uložena u vládních agentur, které mohly na základě soudního příkazu komunikaci dešifrovat. Clipper Chip se setkal s masivním odporem veřejnosti a v roce 1994 publikoval kryptograf Matt Blaze výzkum demonstrující zranitelnost v protokolu, která umožňovala mechanismus *key escrow* obejít. Veřejný odpor nakonec vedl k tomu, že byl projekt v roce 1996 tiše opuštěn. Koncept *key escrow* – tedy existence kopie šifrovacího klíče u „důvěryhodné” třetí strany – se ovšem od této doby opakovaně vrací v různých podobách.

Ideologické zázemí celého odporu vůči regulaci šifrování formovalo hnutí **cypherpunks**, jehož e-mailová skupina byla založena v roce 1992 lidmi jako Eric Hughes, Timothy C. May a John Gilmore. Poté v roce 1993 publikoval Eric Hughes *A Cypherpunk's Manifesto*, kde popisoval, že soukromí ve svobodné společnosti vyžaduje kryptografii a psaní kódu je formou politického aktivismu. Z prostředí cypherpunks vzešla přímo či nepřímo řada technologií – anonymní remailery, Tor, BitTorrent nebo např. Bitcoin.

První fáze Crypto Wars skončila koncem 90. let. V roce 1996 rozhodl federální soud v případě *Bernstein v. United States*, že zdrojový kód je chráněnou formou projevu podle prvního dodatku americké ústavy, a v roce 1999 americká vláda výrazně uvolnila exportní omezení na kryptografický software. Silné šifrování přestalo být klasifikováno jako munice, což přímo umožnilo jeho integraci do webových prohlížečů (přechod z 40bitového na 128bitové a později 256bitové TLS), do operačních systémů (FileVault, BitLocker) i do komunikačních aplikací. Kryptografie se poté více rozšířila do mainstreamu především díky dvěma událostem: již zmiňované vydání Bitcoinu v roce 2009, které ukázalo využití asymetrické kryptografie, a přidání end-to-end šifrování do aplikace WhatsApp v roce 2016 jakožto výchozí stav bez možnosti vypnutí (čímž se odlišuje např. od Telegramu). Šifrovaná komunikace se tak zpřístupnila stovkám milionů uživatelů, kteří o tom ani nemuseli vědět.

Od roku 2014, kdy Apple začal defaultně šifrovat iPhony, probíhá to, co je označováno jako Crypto Wars 2.0. Orgány činné v trestním řízení z různých jurisdikcí opakovaně požadují tzv. „zodpovědné šifrování“ – tedy existenci zadních vrátek pro oprávněné orgány. Jako argument se většinou používá boj proti terorismu a šíření dětské pornografie. Kryptografická komunita na to opakovaně odpovídá, že jakýkoli mechanismus umožňující přístup třetí strany je zranitelností, která může být zneužitelná i útočníkem. V Evropské unii je aktuálně projednáván návrh známý jako Chat Control, který by vyžadoval skenování obsahu zpráv na straně klienta před zašifrováním, což je de facto obcházení E2EE, přestože se formálně nejedná o prolomení samotné šifry. Dalším rozměrem současné debaty je nástup kvantových počítačů, které by mohly v budoucnu prolomit dnes používané asymetrické šifry – v local-first kontextu to znamená, že volba kryptografických primitiv musí počítat s horizontem desítek let dopředu a umožnit případný přechod na tzv. *post-quantum cryptography*. [58]

3.3.2 Identita uživatele

Identita uživatele na webu je dnes fragmentovaná mezi mnoho poskytovatelů. Každý vyžaduje jméno a heslo, což často v praxi vede k opakování stejného hesla. V local-first kontextu nelze použít standardní model centrální databáze uživatelů. Autentizaci lze tedy zajistit buď externím poskytovatelem identity (OAuth2, přihlášení přes Google/GitHub), ale tím vzniká závislost na třetí straně. Čistě local-first přístup proto využívá kryptografické klíče.

Identitou uživatele je tedy kryptografický seed nazývaný **mnemonic**, což je textová reprezentace entropie, ze které lze jednotlivé klíče odvozovat. Mnemonic má obvykle podobu 12 nebo 24 standardizovaných anglických slov a slouží jako lidsky zapamatovatelná možnost pro přenositelnost master klíče. Z mnemonicu lze poté klíče odvozovat pomocí standardu BIP-32 (resp. SLIP-21, což je analogický přístup pro symetrické klíče), které pocházejí z kryptoměnových peněženek. Na základě cesty definované v aplikační logice (např. ["evolu", "owner", "writeKey"]) se z mnemonicu deterministicky odvodí nový klíč, který je kryptograficky nezávislý na klíčích odvozených z jiné cesty.

V podstatě tedy veškerou autentizaci zjednodušujeme na otázku: „Dokáže uživatel rozšifrovat data?“ Dr. Lea Kissner píše: *„Kryptografie je transformace mnoha různých problémů na problém správy klíče – a to je problém mnohem těžší, než si většina kryptografů myslí.“* Tato citace sice může působit jako odrazující, ale v jistém smyslu je i povzbuzující – lze se soustředit především na problém správy klíče a pokud se nám ho podaří vyřešit, vyřešíme s ním i řadu dalších problémů [56].

Problém správy klíče (master-key problem) je inherentně svázán s kryptografickou identitou – existuje jeden kritický údaj, jehož ztráta kompletně odepře uživateli přístup k datům. Základním řešením je fyzická záloha mnemonic fráze na papíře a jeho uložení na bezpečném místě, např. v sejfě. V delším časovém horizontu se ovšem zvyšuje riziko poškození papíru, případně jeho ztráty. Bezpečnější, ale také dražší variantou je proto sada kovových destiček, do kterých lze mnemonic vyryt (produkty jako Cryptosteel nebo Trezor Keep Metal). Tím se sice zvyšuje odolnost vůči ohni, vodě i fyzickému poškození, ale mnemonic je stále koncentrován do jednoho bodu – kdo ho najde, má plný přístup. Ještě pokročilejší možností je tedy Shamir Secret Sharing, který rozdělí tajemství na n částí se zvoleným parametrem $k \leq n$ takovým, že libovolných k částí je dostačující pro rekonstrukci celého tajemství a libovolných $k - 1$ částí **neodhaluje o tajemství vůbec nic**. V praxi se často volí rozdělení na 5 částí s prahem 3 – poté si uživatel může např. dvě části nechat u sebe, ostatní roz distribuovat mezi důvěryhodné osoby a pro rekonstrukci tajemství mu stačí libovolné tři části. [16]

3.3.3 Passkeys a WebAuthn

Předchozí kapitola popisovala řešení pro dlouhodobou úschovu mnemonic fráze. Stále ale zbývá druhá otázka: jak uživatele autentizovat při každodenním používání aplikace, aniž by musel mnemonic zadávat při každém spuštění? Naivní řešení v podobě plaintextového uložení mnemonicu v prohlížeči otevírá dveře XSS útoku. Potřebujeme mechanismus, který mnemonic udrží na zařízení v zašifrované podobě a odemkne ho jen po ověření uživatele – ideálně způsobem, který běžný uživatel zná z ostatních aplikací.

WebAuthn je standard konsorcia W3C, který definuje JavaScript API pro kryptografické přihlašování v prohlížeči. Tvoří webovou vrstvu nad protokoly **FIDO2** a **CTAP2** (Client to Authenticator Protocol). Autentizátor může být buď zabudovaný v zařízení (tzv. platform authenticator), typicky využívající Apple Secure Enclave, Windows Hello či Android TEE – nebo externí (tzv. roaming authenticator), kterým bývá hardwarový klíč jako Trezor nebo YubiKey. **Passkeys** jsou konkrétní kryptografické klíče vytvořené přes WebAuthn, navržené jako náhrada tradičních hesel [59].

Ve standardním client-server modelu se passkey využívá pro digitální podepsání challenge, kterou vygeneruje server a na základě předem uloženého veřejného klíče tak potvrdí identitu uživatele. V local-first nemáme server, který by tuto challenge generoval, proto místo toho passkey využíváme pro **odemknutí lokálně šifrovaných dat**. Místo digitálního podpisu tedy potřebujeme mít možnost z autentizátoru

opakovaně a deterministicky odvodit šifrovací klíč. Existují dvě cesty, jak toho dosáhnout.

První možností je využít pole `user.id`. V původní WebAuthn specifikaci je sice určené pro identifikátor uživatele, ale jedná se o 64-bytový buffer, do kterého lze uložit libovolná data. Můžeme tedy vygenerovat náhodný 32-bytový seed, kterým zašifrujeme databázi a uložíme ho do tohoto pole – při každém přihlášení se poté deterministicky vrátí. Výhodou je podpora napříč všemi prohlížeči, ale tento přístup má bezpečnostní výhrady, jelikož pole `user.id` nebylo navrženo pro úschovu citlivých dat a za určitých okolností může uniknout i bez autentizace uživatele. Jedná se tedy spíš o hack, který je použitelný dočasně, než bude podporována lepší alternativa [60] [61].

Druhou bezpečnější možností je využití **PRF extension** (Pseudo-Random Function), která je v rámci **hmac-secret** součástí specifikace CTAP2. Autentizátor v tomto režimu vystupuje jako deterministická funkce s vlastnostmi random oracle: pro daný vstup (sůl) vždy vrátí stejný pseudonáhodný 32-bytový výstup, který je kryptograficky odvozen z privátního klíče `passkey`. Chováním připomíná BIP-32 (který také deterministicky odvozuje klíče), ale s rozdílem, že u PRF je master klíč uložen uvnitř autentizátoru a **nelze ho přečíst** – existuje pouze uvnitř hardwarového čipu. Nevýhodou je omezená podpora na některých platformách (Windows Hello, Firefox pro Android a iOS). Jedná se o oblast aktivního vývoje – např. Firefox pro Android přidal experimentální podporu v lednu 2026 a v době psaní této práce se postupně dostává do stabilní verze. Podobný vývoj probíhá i u ostatních platform [62]. Třetí možností je využít Large Blob extension, která umožňuje autentizátoru ukládat libovolná data (typicky v jednotkách KB) přímo ke konkrétnímu `passkey`. Na rozdíl od PRF zde nejde o derivaci klíče, ale o přímé úložiště. Hlavní nevýhodou je ještě omezenější podpora napříč platformami než u PRF [63].

3.4 Uživatelská kolaborace

Uživatelskou kolaborací je myšlena schopnost více uživatelů spolupracovat nad sdílenými daty. To se samozřejmě netýká každé local-first aplikace, pro mnoho případů užití (správce hesel, osobní finance) je režim jednoho uživatele dostačující. Uživatelská kolaborace je jedna ze složitějších oblastí local-first a existuje více možných přístupů – proto je nutné její implementaci dostatečně zvážit.

Jeden z možných modelů kolaborace je po vzoru Gitu: uživatelé pracují lokálně na svých kopiích dat a periodicky nahrávají své změny na sdílené úložiště. Spolupráce je asynchronní a pro nahrání změn vyžaduje explicitní akci (`git push`). Druhým modelem je real-time spolupráce např. po vzoru Google Docs, kde je každá změna okamžitě sdílena mezi ostatní uživatele.

S uživatelskou kolaborací přicházejí tři vzájemně provázané problémy: jak řešit konflikty při souběžných úpravách, jak identifikovat uživatele a jak spravovat členství ve skupině uživatelů. První dva problémy byly popsány v předchozích kapitolách, tato kapitola se tedy věnuje převážně správě členství ve skupině.

3.4.1 Členství ve skupině uživatelů

Z hlediska správy přístupu lze popsat dva základní modely skupiny. **Org-owned** model spravuje skupinu skrze centrální Relay, typicky se může jednat o firmu/organizaci. Tento přístup je jednodušší na implementaci, ovšem dává plnou moc do rukou dané organizaci. **Member-owned** model naopak staví všechny členy skupiny do rovnocenné pozice a správa skupiny probíhá na základě konsensu. Tento přístup je více decentralizovaný a umožňuje dosáhnout většího soukromí, ale je složitější na provoz.

Pro **přidání nového člena** je v první řadě nutné vyřešit problém, který se nazývá *bootstrap truth*. Jde o to, že nestačí, aby kandidát svůj veřejný klíč pouze zveřejnil, ale musí také nějak prokázat vazbu mezi tímto klíčem a konkrétní osobou. To se dá delegovat na existující důvěryhodné kanály jako Signal, iMessages nebo např. osobní setkání s naskenováním QR kódu. Operace přidání člena je poté již triviální – po úvodním navázání spojení stačí kandidátovi předat aktuální šifrovací klíč a zapsat ho do seznamu členů.

Naopak **odebrání člena ze skupiny** je složitějším tématem. Zatímco přidání člena distribuuje novou informaci směrem ven, odebrání člena se snaží již distribuovanou informaci získat zpět – a to v local-first prostředí není plně možné, jelikož data už existují v lokální databázi uživatele a neexistuje prostředek, kterým mu je lze odebrat. Při odebrání člena lze tedy zaručit pouze ochranu dat vytvořených v budoucnu. S tím souvisí následující dva pojmy. *Forward-secrecy* znamená, že kompromitace aktuálních klíčů neumožní útočnickovi dešifrovat data z minulosti. Je tedy vyžadována pravidelná rotace klíčů a především likvidace klíčů starých. Nevýhodou je, že se po nějaké době začne ztrácet historie – pro určité oblasti využití to ovšem nemusí být problém. *Post-compromise security* znamená, že pokud útočník kompromituje aktuální klíč, nebude mu fungovat navždy a systém se po určitém čase sám uzdraví. Znovu je tedy nutné pravidelně rotovat klíče a při každé rotaci zamíchat do klíče novou entropii. Uvedené vlastnosti jsou na sobě nezávislé – mohou platit buď obě, jedna z nich, nebo žádná.

Prakticky probíhá odebrání tak, že zbývající členové vygenerují nový šifrovací klíč a distribuují ho všem ostatním členům skupiny kromě odebraného. Nejjednodušší možností je tzv. **envelope encryption** – místo toho, aby se data šifrovala zvlášť pro každého člena se zašifrují pouze jednou pomocí společného symetrického klíče (DEK – Data Encryption Key) a tento klíč se poté samostatně znovu zašifruje pro každého člena skupiny zvlášť (např. pomocí jeho veřejného klíče) a připojí se k datům jako pomyslná obálka. Při rotaci klíčů tak stačí přeshifrovat DEK a není nutné sahat na data. Nevýhodou je, že tento přístup škáluje lineárně – pro skupinu o N členech vyžaduje $\mathcal{O}(N)$ asymetrických šifrování při každé rotaci. Envelope encryption také samo o sobě nezaručuje forward-secrecy a post-compromise security.

3.4.2 Message Layer Security

Alternativou k envelope encryption je **Message Layer Security** (MLS). Jedná se o standard IETF (RFC 9420) pro správu klíčů skupiny, který zaručuje jak post-compromise security, tak forward-secrecy, ovšem na rozdíl od envelope encryption

využívá binární strom (ratchet tree), díky čemuž má operace změny členství pouze logaritmickou složitost.

Každý z uzlů stromu obsahuje vlastní klíčový pár (veřejný a privátní) a jednotliví členové skupiny odpovídají listům stromu. Každý člen zná soukromé klíče pouze na cestě od svého listu ke kořeni stromu, nikoliv klíče sourozenců. Kořenový klíč se poté používá k odvození společného symetrického klíče pro šifrování zpráv. Epocha reprezentuje konkrétní stav skupiny v daném časovém období – každé přidání nebo odebrání člena (které je zabaleno v tzv. commitu) posouvá skupinu do nové epochy. Rotaci epochy může provést libovolný člen skupiny (iniciátor) a probíhá následujícím způsobem. Nejprve vygeneruje iniciátor nový náhodný secret pro svůj list a z něj odvodí sekvenci klíčů na celé cestě ke kořeni stromu. Tyto nové klíče poté zašifruje veřejnými klíči odpovídajících podstromů na opačné straně cesty – takovým způsobem, aby každý člen dokázal dešifrovat pouze klíč toho uzlu, který odpovídá nejbližšímu společnému předku jeho listu a listu iniciátora. Od tohoto uzlu si pak každý člen skupiny odvodí zbytek cesty ke kořeni sám, aniž by mu klíče musely být explicitně posílány. Po aplikaci commitu se také vždy změní hlavní symetrický šifrovací klíč [64].

Standardní MLS využívá jakožto ratchet tree implementaci TreeKEM a ta předpokládá existenci centrálního serveru (Delivery Service), který rozhoduje o pořadí commit zpráv. Pro local-first prostředí je toto problém, a proto vzniká standard **Decentralized MLS** (DMLS), jehož cílem je zbavení se závislosti na centrálním serveru díky konsenzuálnímu rozhodování mezi členy skupiny ohledně pořadí commitů. Ponechává TreeKEM, ale členové skupiny se nejprve dohodnou na tom, který commit je další v pořadí, a až poté ho aplikují. V local-first kontextu může ovšem nastat problém, že by jeden offline uživatel blokoval rotaci epochy pro všechny ostatní [65]. Ink & Switch v rámci projektu Keyhive navrhuje alternativní řešení – nahrazuje TreeKEM za vlastní protokol **BeeKEM**. Ten přepracovává samotnou stromovou strukturu takovým způsobem, že pokud dva členové skupiny provedou souběžné změny, jsou obě dočasně zachovány jakožto konfliktní a po výměně dat algoritmus větve automaticky sloučí. [66] [67]

3.5 Bezpečnostní hrozby

Vzhledem k přesunutí dat a většiny logiky aplikace na klienta jsou i potenciální bezpečnostní hrozby odlišné od client-server aplikací, kde bývá kritickým bodem selhání server. Díky tomu už sice nehrozí hromadný únik dat z centrální databáze, ale stav je místo toho distribuován na zařízení klienta, které se tím stává náchylnějším na útok. V kontextu webu má většinou aplikace omezená práva, kam může přistupovat, a je potřeba ruční schválení práv od uživatele. Kyle Simpson tvrdí, že rozdělovacím faktorem je, pokud si uživatel aplikaci explicitně nainstaluje (ve webovém prostředí u PWA „Přidat na plochu“). Tím dává najevo, že aplikaci důvěřuje a chce jí povolit přístup k pravomocím – v podstatě se tím z pohledu důvěry dostává na úroveň nativní aplikace [16].

OPFS je sice sandboxově izolované na úrovni originu stránky, ale soubory na disku nejsou šifrovány – prohlížeč je pouze obfuskuje, tedy typicky rozdělí jako binární bloky mezi více souborů s náhodnými názvy. Samotný obsah ovšem zůstává totožný, tudíž útočník s přístupem k disku může data SQLite databáze přechíst i mimo kontext prohlížeče. Lze to ověřit jednoduchým Python scriptem, který prochází datový adresář Chrome a v každém souboru hledá binární hlavičku „SQLite format 3“. Nalezené soubory se potom zkopírují mimo OPFS a lze je otevřít standardním SQLite prohlížečem. Z toho důvodu je kromě šifrování dat při přenosu sítí nezbytné šifrovat také samotnou databázi na úrovni disku.

Cross-site scripting (XSS), tedy injekce škodlivého JavaScriptu do běžící aplikace, je další potenciální hrozbou. Zatímco u client-server aplikace se útočník skrze XSS dostane maximálně k session tokenu nebo k aktuálně načteným datům v DOM, u local-first by se mohl dostat přímo k datům – což je další důvod proč ji šifrovat. Šifrovací klíč k databázi by samozřejmě neměl být uložen v localStorage nebo IndexedDB, jelikož ty jsou také součástí vektoru útoku. Pokud je šifrovací klíč vázán na hardwarový autentizátor, útočník musí uživatele přimět k biometrickému ověření, což je viditelná a atypická interakce, která dává uživateli šanci útok odhalit. Základní obranou proti XSS však zůstává striktní *Content Security Policy* a minimalizace závislostí na třetích stranách, případně jejich pečlivý audit. S tím souvisí také tzv. supply-chain útoky – kompromitovaná NPM závislost může do aplikace propašovat škodlivý kód. Příkladem z nedávné doby může být zranitelnost Shai-Hulud ze září 2025 nebo kompromitace balíčku axios v březnu 2026.

Relay server je v local-first architektuře záměrně „slepý“ – neinterpretuje obsah zpráv, pouze je přeposílá – což ho činí zranitelným vůči zaplavení velkým objemem nesmyslných dat. Proti obecnému spamu neexistuje čistě technická obrana a je nutné Relay moderovat nebo omezovat rate limitem na úrovni uživatelů. Závažnější variantou této hrozby je *denial-of-sync* na úrovni samotného synchronizačního protokolu: pokud využíváme RBSR a ta dělí sjednocovaný rozsah záznamů na podrozsahy deterministicky, může útočník s částečnou znalostí datasetu vložit speciálně sestavené záznamy, jejichž hash kolize způsobí, že dva rozdílné rozsahy vyjdou jako shodné. Takové záznamy se přeskočí jako již synchronizované a rozdíly v datech zůstanou trvale neviditelné, případně lze takto odvodit existenci konkrétních záznamů na protější straně [68]. Řešením je *range randomization* – hranice rozsahů se nedělí deterministicky, ale s využitím náhodné soli přidávané do hashovací funkce, takže útočník nemůže dopředu zkonstruovat datovou sadu.

3.5.1 Meltdown a Spectre

Zranitelnosti Meltdown a Spectre (2018) jsou sice hardwarového charakteru a nejsou specifické pro local-first, přesto se hodí je popsat v kontextu webu. Moderní procesory používají techniky spekulativního provádění instrukcí (*speculative execution*) a provádění instrukcí mimo pořadí (*out-of-order execution*) pro minimalizaci počtu provedených cyklů a obecně zrychlení programu. Z toho ovšem plyne potenciální bezpečnostní hrozba: instrukce může být spekulativně provedena ještě dříve, než procesor zkontroluje, že má pro danou akci oprávnění. Před rokem 2018 panoval

konsenzus, že se nejedná o zásadní problém, jelikož data budou stejně nakonec zahozena, až procesor vyhodnotí oprávnění. Jako zásadní se ovšem ukázalo, že procesor po sobě zanechává stopy v cache. Útočník tedy může pomocí chytrého poskládání instrukcí donutit procesor načíst data z jiné části RAM, kam aplikace vůbec nemá přístup, a následně díky měření doby odezvy cache odvodit jejich hodnotu po bytech – jedná se tedy o tzv. *side-channel attack* [69].

Nutnou podmínkou pro provedení tohoto útoku je časovač s přesností na nanosekundy, čehož lze ve webovém prostředí prakticky dosáhnout pouze s využitím `SharedArrayBuffer`. V reakci na objevení zranitelností byl `SharedArrayBuffer` jako hotfix v Mozille i Chromu nejprve zakázán a zároveň došlo k limitaci přesnosti časových API jako `performance.now()`. Později byla přijata opatření izolující obsah každé webové stránky do samostatného procesu (v Chromu tzv. *Site Isolation*, ve Firefoxu *Project Fission*), čímž se útočníkovi výrazně omezily možnosti přístupu k paměti ostatních stránek. Podpora `SharedArrayBuffer` byla následně obnovena, ovšem pod podmínkou nastavení dvou HTTP hlaviček `Cross-Origin-Opener-Policy: same-origin` a `Cross-Origin-Embedder-Policy: require-corp`, které zajišťují, že stránka běží v tzv. *cross-origin izolovaném kontextu* [70].

4 Local-first knihovny

4.1 Přehled ekosystému

V posledních letech vzniká mnoho různých local-first knihoven. Odlišují se hlavně v úrovni abstrakce – od nízkoúrovňových, kde si vývojář řeší perzistenci a transport dat sám, přes reaktivní databáze a synchronizační enginy až po kompletní platformy poskytující vše v jednom balíčku.

V algoritmech pro synchronizaci se vyprofilovaly dva hlavní směry. **Append-log** přístup si zachovává kompletní historii operací a dotaz mezi klientem a serverem zní „pošli všechno po verzi X”. Implementace je jednodušší, objem přenášených dat ovšem roste lineárně s rostoucí historií. **Range-based** přístup místo toho přenáší kompaktní fingerprinty dat a identifikuje pouze rozdíly – díky tomu se data nepřenáší opakovaně a má logaritmickou složitost za cenu mírně složitější implementace.

Komunita okolo local-first software se schází na dvou výročních konferencích – Local-First Conf v Berlíně (od roku 2024) a Sync Conf v San Franciscu (od roku 2025).

CRDT knihovny

Nízkoúrovňové knihovny, které implementují konkrétní CRDT typy – mapy, listy či rich-text. Neřeší identitu, perzistenci, ani transport dat, proto se hodí tam, kde je potřeba CRDT zasadit do již existující architektury. Navzájem se odlišují především programovacím jazykem, velikostí runtime buildu a množstvím integrací s existujícími technologiemi.

Do této kategorie patří např. **Yjs**, **Automerge** a **Loro**.

Reaktivní databáze

Knihovny, které implementují reaktivní databázi nad lokálním úložištěm dat. Volbu CRDT i synchronizační strategie ponechávají na vývojáři – hodí se tam, kde je synchronizace méně důležitá nebo již řešená přes existující backend. Mezi sebou se liší především datovým modelem (dokumentový JSON či relační databáze).

Do této kategorie patří např. **RxDB**, **PouchDB**, **Dexie** a **TinyBase**.

Sync engines (server-authoritative)

Knihovny, které přidávají k již vytvořené databázi synchronizaci mezi klientem a centrálním serverem. Nabízejí snadnou integraci s existujícími relačními databázemi,

nejedná se ovšem o plně local-first přístup, jelikož stále existuje autoritativní server. Mezi sebou se odlišují hlavně množstvím podporovaných databází, strategií řešení konfliktů a monetizačním modelem.

Do této kategorie patří např. **Replicache**, **Zero**, **ElectricSQL**, **PowerSync** a **Triplit**.

Local-first platformy (user-authoritative)

Platformy, které poskytují lokální databázi, synchronizaci dat, identitu a E2E šifrování v jednom celku. Odlišují se použitou transportní vrstvou, datovým modelem a synchronizačním algoritmem.

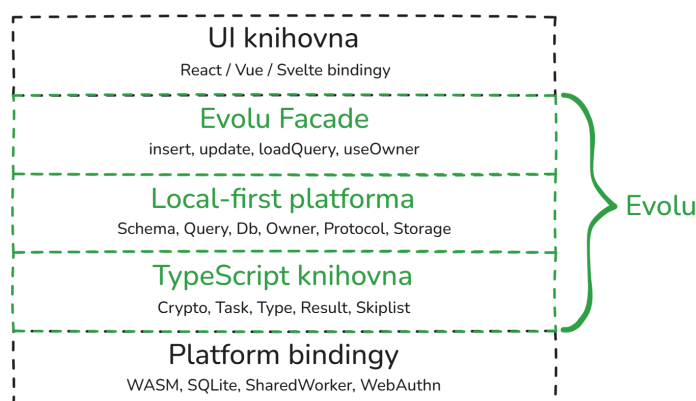
Do této kategorie patří např. **Evolu**, **Jazz.tools**, **DXOS**, **LiveStore** a **Ditto**. [71] [72]

4.2 Knihovna Evolu

Evolu je local-first platforma navržena pro maximalizaci soukromí, jednoduchost použití a zabránění proprietárnímu uzamčení. Skládá se ze dvou částí – první je TypeScript knihovna, která poskytuje obecné návrhové vzory pro typové řízení chyb, dependency injection, validaci, parsování atd. Druhou částí Evolu je knihovna pro tvorbu local-first aplikací, která poskytuje přístup k SQLite na všech platformách, CRDT pro řešení konfliktů pomocí LWW strategie, mechanismus pro šifrovanou synchronizaci dat v reálném čase skrze WebSocket, generický Relay server a typované databázové schéma pomocí Kysely. Evolu také poskytuje konstrukty pro implementaci uživatelské kolaborace, kde je identita uživatele postavena na mnemonicu. Není závislé na zobrazovací vrstvě, lze ho tedy použít s libovolnou UI knihovnou. [73]

4.3 Vnitřní implementace Evolu

Evolu si lze představit jako layer-architekturu s pěti vrstvami nad sebou. Hlavní instanci Evolu si lze představit jako návrhový vzor Fasáda.



Obrázek 4.1: Layer-architektura knihovny Evolu

4.3.1 TypeScript knihovna

Evolu TypeScript knihovna obsahuje návrhové vzory známé především z funkcionálního programování a jazyků jako Haskell či Scala. Implementuje je ovšem v idiomatickém TypeScriptu bez složitých abstrakcí (monády) či pipe operátorů. Knihovna je publikována jako samostatný balíček, lze ji tedy použít i mimo Evolu.

Dependency Injection

Evolu používá Dependency Injection (DI) postavené na nativních konstruktech TypeScriptu místo Inversion of Control (IoC) kontejneru. Dále se vyhýbá vzoru Service Locator – globálně dostupná závislost, která skrývá vazby a ztěžuje testování.

Závislosti jsou místo konkrétních implementací definované jako rozhraní (např. `Time`, `Console`). Funkce je přijímají explicitně jako parametr. Pro oddělení závislostí od datových parametrů se využívá TypeScript technika *currying* – funkce nejprve přijme objekt se závislostmi a teprve druhé volání přebírá vlastní argumenty.

Všechny závislosti se kombinují do jednoho objektu, který se předává napříč kódem a každá funkce z něj použije pouze to, co skutečně potřebuje. Jelikož jde o standardní TS objekt, lze nad ním aplikovat funkce jako `Omit`, `Partial` či využívat spread operátor. Každý balíček s konkrétní implementací pro danou platformu (`@evolu/web`, `@evolu/nodejs`) má definované vlastní závislosti [73].

Evolu Type

Evolu Type je sada nástrojů pro validaci a parsování. Staví na nativních type guardech a klíčových slovech `typeof`, `instanceof`, které rozšiřuje o typované chybové stavy.

Běžné validační knihovny fungují tak, že pokud nastane chyba, vyhodí výjimku (Exception) s textovou hláškou. TypeScript ale neumí staticky typovat výjimky, tudíž volající kód nevidí, jaké všechny chyby mohou nastat. Evolu Type místo toho vrací hodnotu typu `Result`, která zapouzdřuje úspěch i chybu do jednoho datového typu. Každý chybový stav je definován jako samostatný typ, takže kompilátor může vynutit ošetření všech chyb. Při přidání nového chybového stavu také automaticky označí všechna místa volání, kde není pokryt.

Evolu Type podporuje Standard Schema, což zajišťuje kompatibilitu s více než 40 dalšími validačními knihovnami jako např. `Zod` nebo `Effect Schema`. Součástí Evolu Type jsou také tzv. `Branded Types` – slouží pro rozlišení primitivních typů, které by jinak byly zaměnitelné. Umožňují odlišit například `UserId` od `OrderId`, přestože jsou oba stringem. Dále lze vytvořit typy jako `NonEmptyString1000`, `EmailString` či `Kilograms`. `Branded Types` vynucují doménové invarianty na úrovni typového systému a zároveň slouží jako dokumentace – z podpisu funkce je zřejmé, jaký tvar dat očekává [74].

Evolu Task

Evolu Task je abstrakce pro spouštění asynchronních operací využívající typ `Result`. Nativní `Promise` v JavaScriptu trpí stejným problémem jako výjimky, tedy

chybový stav není součástí typu, volající kód tudíž neví, co všechno může selhat. Task je oproti tomu funkce, která vrací `Promise<Result<T, E>>`.

Task je navržen jako *lazy* – na rozdíl od `Promise` se nespustí při vytvoření, ale až při explicitním zavolání. Díky tomu lze využívat operace jako opakované spuštění, odložené spuštění nebo omezení paralelismu pomocí semaforu, aniž by došlo k nechtěnému spuštění. K volání Tasku lze také volitelně přidat `TaskContext`, což ho umožní zrušit pomocí `AbortSignal`. Typickým použitím je řetězení síťových operací, například při volání `fetch` [75].

4.3.2 Ukládání dat

Evolu implementuje CRDT přímo nad SQLite. Ke změnám nedochází přímým zápisem do tabulek, ale skrze CRDT zprávy, které popisují jednotlivé modifikace sloupců a jsou všechny uloženy v centrální systémové tabulce `evolu_message`, ze které lze odvodit aktuální stav aplikačních tabulek. Pro čtení dat lze volat libovolné SQL dotazy bez omezení, zápis dat ovšem musí být vázán na konkrétní tabulku a ID řádku a procházet skrze Evolu funkci `update`, která z SQL dotazu právě generuje CRDT zprávy. Evolu tedy nepovoluje bulk-write operace přímo v SQL, aby se předešlo nechtěnému vygenerování obrovského množství CRDT zpráv – pokud je bulk-write vyžadován, musí být explicitně proveden skrze cyklus na aplikační úrovni.

Schéma je definováno pomocí knihovny Kysely, která poskytuje typovaný SQL query builder. V kombinaci s Evolu Type a Branded Types má vývojář při psaní dotazů automatické doplňování a typovou kontrolu – data jsou po načtení automaticky přetypována a při zápisu i čtení validována proti schématu. Ke každé tabulce Evolu automaticky přidává systémové sloupce `createdAt`, `updatedAt`, `isDeleted` a `ownerId`. Schéma je append-only a bez verzování; všechny sloupce kromě `id` jsou nullable. Jedná se o podobný přístup, jako používá např. GraphQL [76] a vychází z předpokladu, že data mají dlouhou životnost a musí být dopředu i zpětně kompatibilní. Migrace se neprovádějí jako destruktivní změny schématu, ale inkrementálně – nové sloupce se přidávají a transformace starých dat probíhá v aplikační vrstvě.

Zápisový tok probíhá následovně: Evolu shromáždí mutace do pole a odešle ho ke zpracování do Web Workeru. V rámci workeru proběhne typová validace proti schématu a z každé mutace vznikne CRDT zpráva, která obsahuje identifikaci tabulky, řádku a sloupce, novou hodnotu a unikátní HLC timestamp. Aplikace zpráv do databáze probíhá v transakcích. Mutace se řadí do mikrotaskové fronty, což umožňuje dávkové zpracování – více mutací v rámci jednoho tiky event loopu se zpracuje v jedné transakci. Totožný mechanismus platí i pro CRDT zprávy, které jsou přijaté z jiných zařízení.

Díky reaktivním dotazům na úrovni SQL odpadá potřeba dalších knihoven pro správu stavu jako Redux, Pinia či Svelte Store. Lokální SQLite je dostatečně rychlé na to, aby reaktivní dotazy stíhaly běžné UI interakce – data se přímo z databáze dostávají do UI komponent, bez mezivrstvy udržující stav v paměti JavaScriptu.

Evolu poskytuje také local-only tabulky (jejich název začíná podtržítkem), které nepodléhají CRDT mechanismu a nesynchronizují se. Díky tomu nad nimi lze provádět skutečné mazání dat a teoreticky také bulk-write. Slouží typicky jako úložiště

pro dočasná data, která budou poté přesunuta do synchronizované tabulky, případně data specifická pro dané zařízení.

4.3.3 Přenos dat

Evolu původně při synchronizaci dat využívalo Merkle Trees pro evidenci CRDT zpráv, které dané zařízení obsahuje. To se ovšem u větších datasetů ukázalo jako nevhodně škálující, jelikož vložení nového záznamu mění hash na celé cestě od listu ke kořeni. RBSR tento problém nemá – nepracuje s datovou strukturou obsahující celou historii, ale počítá hashe rozsahů dynamicky dle aktuálního stavu dat.

Naivní implementace RBSR v aplikační vrstvě by však při každém kroku rekurze vytvářela nový databázový dotaz, což by znamenalo velké množství meziprocesových roundtripů. Evolu proto implementuje RBSR přímo v SQL pomocí Common Table Expressions (CTE) – rekurzivních dotazů běžících uvnitř SQLite virtuálního stroje, díky nimž je SQL turingovsky kompletní. Algoritmus tak běží co nejbližší datům a výsledkem jediného SQL dotazu jsou přímo záznamy připravené k synchronizaci.

Referenční implementací RBSR je protokol Negentropy, který si vyměňuje pouze identifikátory chybějících záznamů a iniciátor se až následně rozhoduje, zda data chce stáhnout nebo nahrát [68]. Synchronizační protokol Evolu má podobný princip, ale data posílá rovnou jako součást odpovědi (eager sync) – tím se snižuje počet roundtripů za cenu mírně vyššího datového objemu. Podobný princip lze nalézt také v nástroji rsync od autorů SQLite [77].

Evolu používá také karanténu zpráv. Pokud klient přijme CRDT zprávu odkazující na tabulku nebo sloupec, který v aktuálním schématu neexistuje (typicky z novější verze aplikace), Evolu ji nezahodí, ale uloží do karantény. Po aktualizaci schématu se karanténní zprávy automaticky aplikují. Tím je zajištěna dopředná kompatibilita mezi verzemi běžícími na různých zařízeních stejného uživatele.

Evolu Protocol

Evolu Protocol je binární synchronizační protokol pro standardizovaný formát výměny dat. Umožňuje buď synchronizaci mezi klientem a Relay serverem nebo vzájemně mezi dvěma klienty (P2P) či Relay servery. Protokol je optimalizován na co nejmenší datový objem a nejvyšší rychlost – z toho důvodu využívá binární formát místo JSON. Základně je navržen pro SQLite, ale teoreticky ho lze rozšířit také na libovolnou jinou databázi [78].

Každá zpráva protokolu má hlavičku – ta obsahuje verzi protokolu, OwnerId, typ zprávy, případně šifrovací klíč pro zápis – a dvě volitelné sekce pro Messages a Ranges. Sekce Messages jsou samotná data k synchronizaci ve formě šifrovaných CRDT zpráv. Sekce Ranges naopak obsahuje seznam rozsahů a pro každý rozsah jeho hash – slouží jako kompaktní reprezentace stavu databáze pro RBSR. Příklad jedné zprávy protokolu se nachází na obrázku 4.2.

Jednotlivé zařízení vystupují ve dvou rolích: **iniciátor** (typicky klient) a **neiniciátor** (typicky Relay). Iniciátor zahajuje výměnu zprávou typu Request, kde pošle buď data nebo reprezentaci databáze pro RBSR (Ranges). Neiniciátor vždy

odpovídá zprávou typu Response, a to i v případě, že nemá nová data k poslání – prázdná odpověď slouží k signalizaci, že synchronizace je dokončena. Třetím typem zprávy je Broadcast, kterým Relay rozesílá nové změny všem klientům, kteří jsou pro daného Ownera přihlášení k odběru změn.

0	59	193	30	13	197	129	241	80	15	255	45	234	249	223	59	136
0	1	2	31	153	253	156	250	238	50	128	220	15	246	3	165	3
194	1	183	1	157	1	168	1	180	1	165	1	15	247	2	219	64
252	1	179	1	203	2	193	1	227	1	200	1	222	150	18	226	

	Verze protokolu
	OwnerId
	Sync metadata
	Timestampy (zakódované pomocí RLE)

Obrázek 4.2: Struktura zprávy v Evolu Protocol [78]

4.3.4 Relay

Relay je minimální server zajišťující synchronizaci a zálohování dat mezi klienty. Je aplikačně nezávislý a neřeší žádnou business logiku – pravidla se kontrolují na straně klienta. Relay také neudrhuje stav synchronizace jednotlivých klientů, ukládá pouze minimální konfiguraci pro řízení přístupu. Díky tomu je horizontálně škálovatelný a snadno zaměnitelný – uživatel může bez ztráty dat kdykoliv přejít k jinému Relay. Je distribuován ve formě Docker image, vlastní Relay může tedy provozovat tvůrce aplikace ve formě placené synchronizace, koncový uživatel pro vlastní účely nebo třetí strana jako veřejnou službu.

Pro zápis do Relay je nutné předložit **WriteKey**, který Relay validuje vůči danému **OwnerId**. Dále jsou vynucovány rate limity, případně maximální velikost uložených dat. Čtení obsahu žádné autorizaci na úrovni Relay nepodléhá – autorizace se rovná schopnosti dešifrovat zprávy pomocí **EncryptionKey**, který Relay nezná. [73]

4.3.5 Owner API

Owner je v Evolu abstrakce reprezentující vlastnictví dat. Každá CRDT zpráva je přiřazena určitému Ownerovi, což umožňuje řízení přístupu, částečnou synchronizaci a skutečné mazání dat – individuální CRDT zprávy z principu mazat nelze, ale celého Ownera lze prostě přestat synchronizovat. Owner není nutně reprezentací fyzické osoby; může se jednat o logickou podmnožinu dat v rámci aplikace.

Technicky je každý Owner sadou kryptografických klíčů deterministicky odvozených z **OwnerSecret** algoritmem SLIP-21. Skládá se z **OwnerId** (globálně unikátní

identifikátor), **EncryptionKey** (symetrický šifrovací klíč) a **OwnerWriteKey** (rotovatelný klíč pro zápis). Jednotlivé typy Ownerů se liší hlavně tím, které klíče obsahují – z pohledu Evolu jde o stejný objekt, rozdíly jsou spíše v doménovém odlišení use-cases. Existují čtyři základní varianty:

- **AppOwner** – hlavní Owner aplikace, vzniká při prvním spuštění; slouží pro koordinaci dat a správu dalších Ownerů.
- **ShardOwner** – rozděluje data na logické oddíly pro částečnou synchronizaci nebo selektivní mazání.
- **SharedOwner** – kolaborativní zápis pro více uživatelů; obsahuje **WriteKey**, který lze sdílet.
- **SharedReadOnlyOwner** – varianta **SharedOwnera** bez **WriteKey**, určená pouze pro čtení.

Nejjednodušší možný přístup je držet všechna data aplikace v rámci **AppOwner**. To je pro mnoho případů použití dostačující, znamená to ovšem, že se všechna data synchronizují mezi všemi zařízeními a nelze je skutečně mazat. Sharding do více Ownerů tyto problémy řeší: v **AppOwner** jsou uloženy jen odkazy na další Ownery a samotný obsah lze poté synchronizovat selektivně nebo smazat (přestat ho synchronizovat).

Příkladem z praxe může být aplikace na projektové řízení, kde má každý uživatel vlastní **AppOwner** se seznamem dokumentů a každý dokument je samostatný **ShardOwner**. Naopak pro více uživatelů, např. v chatovací aplikaci, odesílatel vytvoří nový **SharedOwner**, zapíše do něj zprávy a mnemonic předá příjemci. Ten si ho aktivuje pomocí metody **useOwner** a Evolu automaticky začne synchronizovat příslušná data. Pro lepší správu skupiny je do budoucna v Evolu plánováno rozdělení **SharedOwner** na dvě varianty: **EccSharedOwner** (eliptické křivky, kompaktní podpisy, není post-quantum safe) a **MlsSharedOwner** (MLS podpisy, datově náročnější, odolný vůči kvantovým útokům i scénářům typu „harvest-now-decrypt-later“). [73]

4.3.6 Bezpečnost a soukromí

Všechna data v Evolu jsou automaticky šifrovaná – jak v databázi na úrovni disku, tak při přenosu a uložení na Relay. Relay server je slepý vůči obsahu a vidí pouze **OwnerId**, timestamp a zašifrovaný blob dat.

Timestamp nemá smysl skrývat, jelikož útočník by ho stejně dokázal odvodit z časování paketů – a zároveň je potřebný k synchronizaci. Pro dosažení ještě většího soukromí (skrytí časových vzorců aktivity) lze na úrovni aplikace použít write queue: zápisy jdou nejprve do nesynchronizované tabulky a v náhodných intervalech se přesouvají do synchronizované. Cenou za to je ztráta kolaborace v reálném čase. Druhou možností pro anonymizaci aktivity je přidávat uměle vytvořený šum neexistujících timestampů.

Šifrování chrání pouze obsah dat, nikoliv metadata. Samotná velikost datového objemu může napomoci k prozrazení obsahu (např. identifikace streamovaného videa

podle velikosti paketů). Evolu proto využívá PADMÉ pro vyplnění binární zprávy a skrytí reálné délky dat při nejvýše 12 % datové režii [79].

Evolu generuje pro každé zařízení `NodeId` – náhodnou 64bitovou hodnotu, která je součástí HLC timestampu. Pokud by tedy dvě zařízení vygenerovala stejná ID, byl by je problém odlišit. Jedná se ovšem o velmi krajní případ – v první řadě by muselo jít o dvě zařízení v rámci jednoho Ownera a s 64bity entropie je to téměř nemožné. Viz šance na kolizi dvou ID pro N zařízení (narozeninový paradox): [80] [81]

- 1 000 zařízení → 0,000000000000271 %
- 1 000 000 zařízení → ~0,00000271 %
- 135 000 000 zařízení → ~1 %
- 1 000 000 000 zařízení → ~3 %
- 4 290 000 000 zařízení → ~50 %

4.4 Open-source rozšíření Evolu

Přidání podpory pro Vue

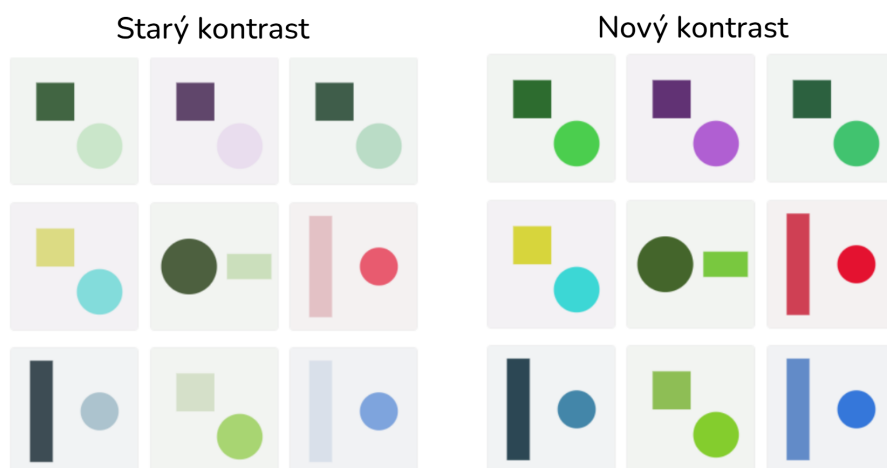
Evolu mělo dosud klientské balíčky pouze pro React a Svelte, chyběla integrace s Vue. Vytvořil jsem nový balíček `@evolu/vue`, který poskytuje funkce pro reaktivní dotazy a přístup k Ownerovi nad hlavní Evolu instancí. Součástí je dále ukázková aplikace `vue-vite-example` a přidání odpovídající sekce do Quickstart sekce v dokumentaci. Implementace navazovala na starší neuzavřený pull request od jiného autora – z něj jsem převzal výchozí strukturu, ale přepracoval jsem ji na aktuální verzi Evolu.

Přidání nových testů

Evolu Type, který poskytuje runtime validaci datových typů, neměl pokryté některé větve kódu. Doplnil jsem jednotkové testy pro primitivní, doménové i kompozitní typy a pro všechny větve formátovače s chybovými hláškami. Pokrytí kódu se tím zvedlo ze 72 % na 98 %. Nepokryté zůstaly už jen polyfilly pro konverzi `base64/Uint8` určené pro starší prohlížeče, které v Node prostředí Vitestu nešlo rozumně otestovat.

Úprava kontrastu u Sutnar Identicon

Sutnar je jeden ze stylů identiconů zabudovaných v Evolu – generuje deterministický SVG avatar z `OwnerId`. Při použití ve menších velikostech (typicky jako avatar v hlavičce) jsem ovšem narazil na to, že někdy generoval barvy, které byly obtížně viditelné na světlém pozadí. Upravil jsem parametry generátoru tak, aby měl avatar vyšší kontrast vůči pozadí. Ukázka výstupu po úpravě je na obrázku 4.3.



Obrázek 4.3: Sutnar Identicon po úpravě kontrastu

Úpravy v dokumentaci

V dokumentaci Evolu jsem provedl úpravy drobných chyb a jazykové korekce. Indikátor aktivní stránky v levém sidebaru nefungoval pro vnořené cesty (např. `/api-reference/react`), protože porovnání bylo přes přesnou shodu – doplnil jsem `pathname.startsWith()` pro vnořené cesty a ponechal přesnou shodu pro top-level (jinak by u cesty `/docs` vyšla shoda všude). Dále jsem opravil hardcoded odkazy na blog, doplnil chybějící nadpis v sekci Showcase a přeformuloval popis AppOwner.

Průzkum PRF rozšíření

PRF je rozšíření WebAuthn, které se používá pro deterministické odvození symetrického klíče. V Evolu se s ním do budoucna počítá v komponentě LocalAuth pro šifrovací klíč databáze. Provedl jsem průzkum tří oblastí – synchronizaci passkeys napříč platformami (iCloud Keychain, Google/Microsoft Password Manager, Windows Hello), podporu PRF v jednotlivých kombinacích OS + prohlížeče a klasifikaci chybových stavů WebAuthn procedury. Jedním je `NotAllowedError`, který může nastat z několika různých důvodů – a jediným způsobem pro jejich odlišení je časová heuristika. Výstupem jsou poznámky a referenční implementace PRF rozšíření v TypeScriptu.

5 Implementace local-first aplikace

5.1 Obecná struktura

Local-first aplikace se obecně většinou skládá z následujících vrstev. Základem je **datové schéma**. Vývojář v aplikaci definuje strukturu svých tabulek skrze typový systém dané knihovny a nad nimi poté volá reaktivní dotazy. Výsledky těchto databázových dotazů jsou přímo napojené na komponenty v uživatelském rozhraní. „Reaktivní“ znamená, že se knihovna automaticky postará o to, aby se dotaz zavolal znovu při změně dat – tím se také překreslí uživatelské rozhraní.

Uživatelské rozhraní aplikace se liší dle cílové platformy. Pro nativní mobilní aplikace lze využít např. UIKit nebo Jetpack Compose, pro desktopové aplikace Qt nebo MAUI, případně React Native nebo Flutter pro cross-platform přístup. V prostředí webu se typicky používá některá z komponentových UI knihoven jako React, Vue nebo Svelte, které umožňují deklarativně popsat uživatelské rozhraní jako funkci stavu. Právě díky této vlastnosti lze implementovat reaktivní dotazy.

Další vrstvou je **identita uživatele**. Pokud je odvozena z kryptografického klíče, může být reprezentována jako mnemonic fráze o 24 slovech. Aplikace poté obsahuje místo, kde si uživatel může frázi zobrazit a na jiném zařízení ji zadat pro obnovu profilu a synchronizaci dat. Pokud je tato fráze také uložena v nějakém z webových úložišť (IndexedDB, OPFS), je vhodné data zašifrovat a použít WebAuthn k odemknutí např. pomocí biometriky.

Se synchronizací přímo souvisí **UI pro zobrazení stavu synchronizace**. Uživatel by měl vidět, zda je aplikace online a synchronizovaná, zda právě synchronizace probíhá, nebo zda pracuje offline s čekajícími změnami. Typickou implementací může být barevný indikátor v hlavičce stránky.

Některé aplikace potřebují **UI pro ruční řešení konfliktů**. CRDT řeší některé konflikty automaticky, ale u složitějších dat (typicky textové povahy) může být automatické sloučení se strategií Last Write Wins nedostatečné. V takových případech aplikace detekuje konflikt a nabídne uživateli výběr mezi verzemi, případně jejich ruční sloučení (podobně jako řešení konfliktů u Gitu).

Další zmíněné vrstvy aplikace již nejsou specifické pro local-first. **Uživatelské nastavení** (barevný motiv, jméno uživatele atd.) může využívat standardní SQL tabulku stejně jako ostatní data, čímž se automaticky synchronizuje mezi všechna zařízení. **Import a export dat** ve strojově čitelném formátu se hodí především, pokud se data přímo nevážou k dané aplikaci a lze je využít i mimo ni [82]. Pro

aplikaci také může být vyžadován multijazyčný překlad (i18n) nebo globální error callback se zobrazením notifikací.

5.2 Zaměření ukázkové aplikace

Ukázková aplikace slouží pro evidenci výpůjček laboratorního vybavení (měřících přístrojů, náradí či vývojových desek) na univerzitě. Řeší problém, kdy si uživatel chce půjčit předmět, ale neví, kdo ho zrovna má půjčený a na jak dlouho. Pro demonstraci local-first přístupu se aplikace hodí z následujících důvodů: data lze přirozeně shardovat dle místností (každá místnost je samostatný kontext bez vazby na ostatní), offline dostupnost dat dává v této oblasti smysl (potenciálně špatný signál v některých místnostech) a uživatelé si v rámci jedné místnosti navzájem důvěřují (není třeba pokročilého systému rolí). Aplikace aktuálně není zamýšlena pro reálné nasazení do praxe, slouží pouze pro demonstraci local-first principů.

5.3 Použité technologie

Vue.js

Jako UI framework se využívá Vue 3 s Composition API a syntaxí `<script setup>`. Vue používá reaktivní model postavený na Proxy objektech, které u šablony sledují závislosti na reaktivních hodnotách a překreslují ji v případě změny. Aplikace je psána v TypeScriptu se `strict` módem.

Evolu

Local-first vrstvu zajišťuje knihovna Evolu. Více je popsána v předchozí kapitole. Mezi hlavní důvody pro její volbu patří automatické end-to-end šifrování, identita založená na mnemonic frázi, synchronizační Relay server nezávislý na aplikaci a MIT licence.

Vitest a Playwright

Pro jednotkové testy slouží Vitest, který sdílí konfiguraci s build nástrojem Vite a nevyžaduje samostatné nastavení. Pro end-to-end testování se používá Playwright, který umožňuje testovat aplikaci v různých prohlížečích.

Ostatní nástroje

Pro stylování komponent slouží utility-first CSS framework Tailwind CSS, díky kterému lze vzhled popisovat přímo v šablonách bez samostatných CSS souborů. Pro kontrolu kvality kódu se využívají tři nástroje – Prettier pro formátování, oxlint jako rychlý linter (napsaný v Rustu) a ESLint pro pravidla více specifická pro Vue a TypeScript. Jako build nástroj a dev server slouží Vite.

5.4 Datové schéma

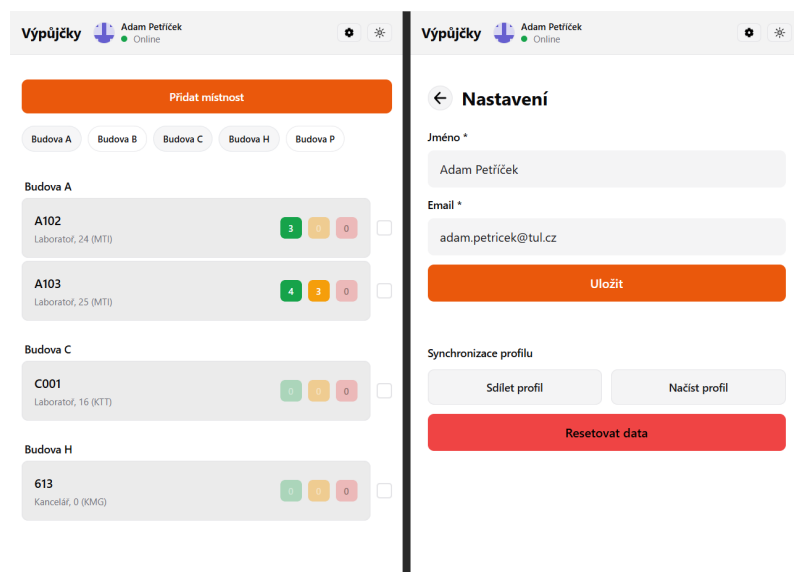
Pro integraci s Evolu je nejprve potřeba definovat datové schéma. Databázové tabulky se popíší pomocí datových typů z Evolu Type a lze využít i Branded Types pro dodatečné odlišení primitivních typů (`NonEmptyString1000`) a primárních klíčů (`Evolu.id('EntityName')`). Po definici schématu ho stačí společně s mnemonicem uživatele a adresou Relay serveru předat funkci `createEvolu`, která vytvoří hlavní Evolu instanci. Tím je v podstatě celá local-first vrstva z pohledu vývojáře nakonfigurována – Evolu zajistí vytvoření SQLite databáze a navázání WebSockets spojení pro synchronizaci, poté už stačí zapisovat data skrze funkci `update`, která je dostupná na hlavní Evolu instanci. Níže se nachází příklad kódu z ukázkové aplikace.

```
1  const Schema = {
2    profile: {
3      id: Evolu.id('Profile'),
4      displayName: Evolu.NonEmptyString100,
5      email: Evolu.nullOr(Evolu.NonEmptyString100),
6      theme: Evolu.nullOr(Evolu.NonEmptyString100),
7    },
8    room: {
9      id: Evolu.id('Room'),
10     name: Evolu.NonEmptyString100,
11     building: Evolu.nullOr(Evolu.NonEmptyString100),
12   },
13   item: {
14     ...
15 }
16
17 const owner = createAppOwner(mnemonicToOwnerSecret(userMnemonic));
18
19 const evolu = createEvolu(Schema, {
20   appName: AppName.orThrow('vypujcky'),
21   appOwner: owner,
22   transports: [
23     createOwnerWebSocketTransport({
24       url: RELAY_URL,
25       ownerId: owner.id,
26     }),
27   ],
28 });
```

5.5 Popis funkcí

Domovská stránka

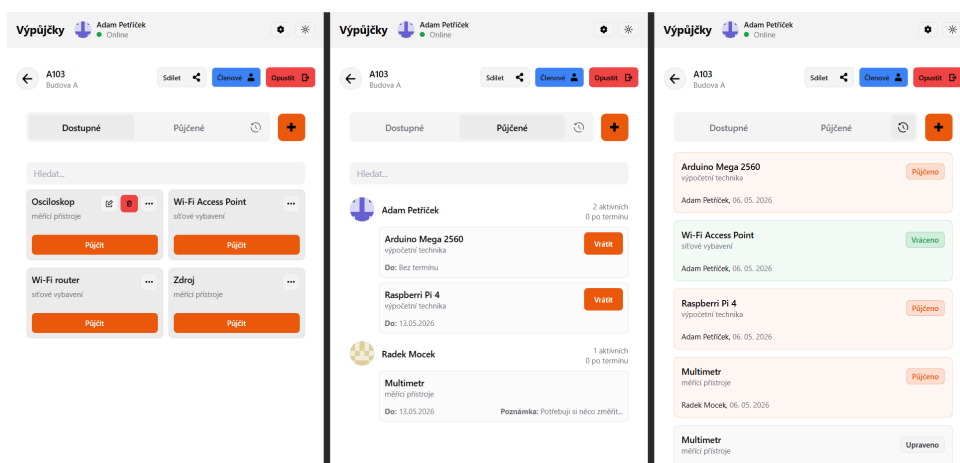
Domovská stránka ukázkové aplikace obsahuje hlavičku s uživatelským jménem a avatarem (identicon), indikátor stavu připojení k internetu a průběhu synchronizace, přepínač barevného režimu a nastavení. Prvním nutným krokem je v nastavení vyplnit jméno a e-mailovou adresu do profilu uživatele. Dále se na domovské stránce nachází seznam místností, ke kterým je uživatel připojen, s možností je filtrovat dle budovy.



Obrázek 5.1: Domovská stránka ukázkové aplikace a stránka s nastavením

Místnosti, sdílení a správa předmětů

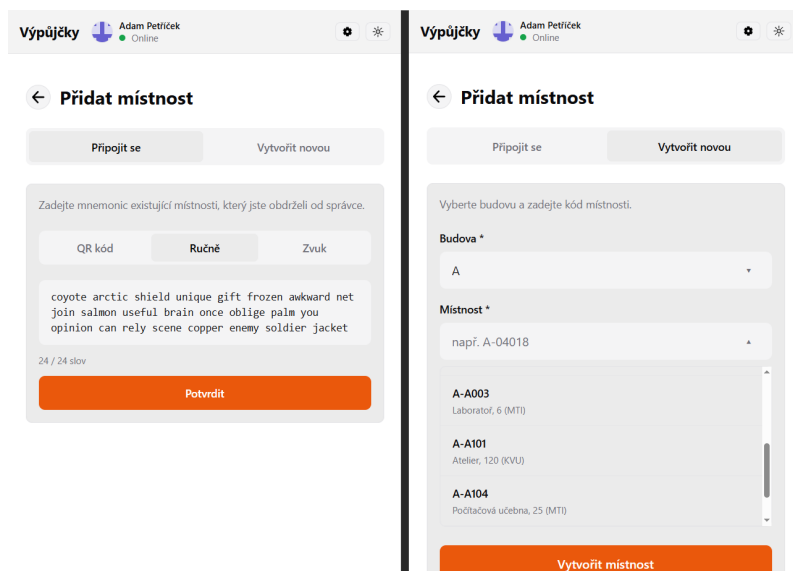
Místnost je základním konceptem aplikace. Pokrývá jeden laboratorní prostor a je reprezentována mnemonicem. Uživatel může buď vytvořit novou místnost (s využitím našeptávače názvů budov a místností z TUL), nebo se připojit k existující místnosti zadáním mnemonicu. Jakmile je uživatel členem, může přístup sdílet s ostatními prostřednictvím QR kódu, ručního zadání nebo zvukového přenosu. Aplikace podporuje také hromadné sdílení – do jednoho QR kódu lze zakódovat více mnemoniců najednou. V detailu místnosti lze poté vytvářet konkrétní předměty (laboratorní vybavení) a provádět jejich vypůjčky. Zobrazení je rozděleno do tří záložek: dostupné předměty, aktuálně vypůjčené předměty a historie všech akcí provedených v dané místnosti.



Obrázek 5.2: Detail místnosti s přehledem dostupných a vypůjčených předmětů a historií operací

Identita uživatele

Identita uživatele je odvozena z jeho osobního mnemonicu (AppOwner), který se vygeneruje při prvním spuštění aplikace. Z něj se deterministicky generuje identicon pro vizuální identifikátor uživatele v rámci místností. V nastavení lze napříč zařízeními synchronizovat celý uživatelský profil – včetně osobních údajů a všech připojených místností. Možnosti sdílení mnemonicu jsou totožné jako u místností. Tlačítko „Resetovat data” vygeneruje nový mnemonic a smaže všechna lokální data.



Obrázek 5.3: Přidání místnosti s možností připojit se k existující nebo vytvoření nové

5.6 Příklad použití v praxi

Pro ilustraci typického použití ukázkové aplikace uvažujeme místnost s laboratorním vybavením na univerzitě. Její správce nejprve v aplikaci vytvoří novou místnost a vyplní v ní seznam dostupných předmětů. Technicky tím v Evolu vzniká **SharedOwner** s náhodně vygenerovaným sdíleným mnemonicem.

Předpokladem je, že má uživatel vyplněný profil (jméno a e-mail). Ten vzniká při prvním spuštění aplikace a technicky se jedná o **AppOwner** s náhodně vygenerovaným osobním mnemonicem. Pro připojení k místnosti si uživatel naskenuje QR kód obsahující mnemonic, který mu poskytl správce (případně ho vloží ručně). Tím si uživatel uloží daný **SharedOwner** do databáze a naváže pro něj WebSocket spojení s Relay serverem, čímž se mu stáhne seznam místností a předmětů.

Poté uživatel klikne na tlačítko „vypůjčit předmět” a vyplní předpokládané datum vrácení. Tím se technicky vytvoří CRDT zpráva (obsahující databázovou mutaci a HLC timestamp), která se uloží do lokální databáze. Tato zpráva je zároveň odeslána na Relay, který ji broadcastuje všem klientům přihlášeným k odběru daného Ownera. Pokud je druhý uživatel stejné místnosti online, změnu vidí téměř

okamžitě. Pokud je offline a vypůjčí si stejný předmět jako první uživatel, vzniknou dvě paralelní výpůjčky. Předmět je znovu označen jako dostupný teprve po vrácení obou výpůjček.

5.7 Testování

Jednotkové testy zajišťuje **Vitest**, který používá testovací prostředí **jsdom** – simulovaný DOM spuštěný v Node.js bez skutečného prohlížeče. V ukázkové aplikaci jsou jednotkové testy použity z většiny pro tzv. čisté funkce (pure functions), které neobsahují žádné side effecty. Jedná se především o pomocné funkce pro např. převod vstupů na Branded Types, pro parsování parametrů Vue Routeru nebo pro vyhledávání v seznamu místností. Testy pokrývají hraniční případy jako prázdné řetězce, **null** či neznámé kódy budov. Dále je testována i hlavní komponenta **App.vue**, kde jsou mockovány závislosti na Evolu instanci a Vue Router.

Pro end-to-end testy se v ukázkové aplikaci používá **Playwright** a spouštějí se ve dvou prohlížečích – Chromiu a Firefoxu. Pokrývají čtyři oblasti: načtení kostry aplikace a přítomnost navigačních prvků v hlavičce, seznam místností na domovské stránce, stránku přidání místnosti a profilu uživatele.

5.8 Build a deployment

Jako build nástroj slouží **Vite**, který zkompiluje TypeScript, zpracuje Vue komponenty se `<script setup>` syntaxí a vygeneruje statické soubory do složky **dist/**. Výsledný build je standardní JavaScript SPA bez vykreslování na serveru, díky čemuž pro hostování stačí libovolný statický hosting.

Ukázková aplikace je hostována na **Cloudflare Pages**, který je přímo napojen na GitHub repozitář. Při každém pushi do hlavní větve Cloudflare automaticky spustí build a pokud projde, provede nasazení do produkce. V aplikaci existuje také **GitHub Actions** pipeline, která se volá při pushi do libovolné větve a spouští pouze typovou kontrolu a testy. Použitý package manager je **pnpm**, který má v testovacím prostředí zamčené verze závislostí.

Hosting Cloudflare Pages byl zvolen z toho důvodu, že je zdarma a pro potřeby ukázkové aplikace obsahuje všechny potřebné funkce – především vestavěnou podporu pro SPA. Na jiném statickém hostingu (např. GitHub Pages) se totiž může stát, že přímý přístup k URL např. `/rooms/add` skončí na hostingu chybou 404, jelikož soubor na dané cestě fyzicky neexistuje. Lze to obejít tak, že CI při buildu zkopíruje **index.html** jako soubor **404.html** (který GitHub Pages vrací při neznámé cestě), čímž proběhne načtení Vue Router a cesta se vyhodnotí správně.

6 Závěr

Tato diplomová práce se zabývala local-first přístupem a jeho praktickou implementací v prostředí webu. V rámci řešerše byly nejprve obecně popsány vlastnosti local-first software, jeho důsledky pro vývojáře a uživatele, historický kontext a možný budoucí vývoj. Dále řešerše popsala konkrétní technologie pro tvorbu local-first aplikací v oblastech ukládání a synchronizace dat, šifrování a uživatelské identity a kolaborace mezi uživateli. Následně byl proveden přehled ekosystému local-first knihoven rozdělením do čtyř kategorií dle úrovně abstrakce a detailní popis vnitřní implementace knihovny Evolu.

Téma local-first kombinuje jak starší, tak novější technologie a relevantní informace jsou rozprostřeny napříč mnoha zdroji – i velké jazykové modely je zatím mají problém spolehlivě shrnout bez velmi detailního promptu. Jedním z přínosů práce je proto zasazení local-first do kontextu webu, zmapování ekosystému a shrnutí použitých zdrojů na jednom místě, což může sloužit jako vstupní bod pro další zájemce o tuto problematiku. Hlubší pochopení technologií bylo zároveň potřebné pro kontribuci do Evolu.

Praktická část práce zahrnuje open-source příspěvky do knihovny Evolu a implementaci ukázkové aplikace. Jedná se o aplikaci pro evidenci výpůjček laboratorního vybavení na univerzitě. Byla postavena na frameworku Vue 3 a knihovně Evolu. Demonstruje principy local-first přístupu založením uživatelské identity na 24-slovném mnemonicu, ukládáním dat v šifrované SQLite databázi a jejich synchronizací v reálném čase skrze generický Relay server. Sdílení přístupu k jednotlivým místnostem je možné přes QR kód, audio přenos nebo ruční přepis. Aplikace běží na Cloudflare Pages s automatizovaným nasazením přes GitHub Actions.

Práci by bylo možné rozšířit v několika směrech. Ukázková aplikace by mohla být přenesena na další platformy (Android, iOS, desktop). Původním záměrem bylo aplikaci vytvořit ve frameworku Lynx, který umožňuje webový kód zkompilovat cross-platform nativně. Jedná se ovšem o relativně nové řešení s horší dokumentací a vytvoření adaptéru do Evolu se ukázalo složitějším.

V rámci samotné knihovny Evolu se nabízí rozšíření ve dvou oblastech. Prvním je přidání vrstvy asymetrické kryptografie (např. eliptické křivky nebo MLS) ke `SharedOwner`, což by umožnilo lepší správu skupiny a odlišení rolí (administrátor, uživatel). Druhou oblastí je dokončení komponenty `LocalAuth`, konkrétně implementace WebAuthn PRF pro deterministické odvozování šifrovacího klíče k lokální databázi z hardwarového autentizátoru.

Seznam použité literatury

1. KLEPPMANN, Martin; WIGGINS, Adam; HARDENBERG, Peter van; MCGRANAGHAN, Mark. Local-first software: You own your data, in spite of the cloud. *Ink & Switch* [online]. 2019 [cit. 2026-05-04]. Dostupné z: <https://www.inkandswitch.com/essay/local-first/>.
2. ARK BUILDERS. *How Local-First Development Is Silently Fixing Everything Wrong With Cloud Software* [online]. 2024-11. [cit. 2026-05-04]. Dostupné z: <https://ark-builders.medium.com/what-is-local-first-ef6e569b33ee>.
3. INDIEWEB. *silo* [online]. [cit. 2026-05-04]. Dostupné z: <https://indieweb.org/silo>.
4. W3TECHS. *Usage Statistics and Market Share of Cloudflare* [online]. [cit. 2026-05-04]. Dostupné z: <https://w3techs.com/technologies/details/cn-cloudflare>.
5. MATTHEW PRINCE. *Cloudflare outage on November 18, 2025* [online]. 2025-11. [cit. 2026-05-04]. Dostupné z: <https://blog.cloudflare.com/18-november-2025-outage/>.
6. DANE KNECHT. *Cloudflare outage on December 5, 2025* [online]. 2025-12. [cit. 2026-05-04]. Dostupné z: <https://blog.cloudflare.com/5-december-2025-outage/>.
7. SANDRO MAGLIONE. *Notes on local first* [online]. [cit. 2026-05-04]. Dostupné z: <https://sandromaglione.com/articles/notes-on-local-first>.
8. DIVYA. *Local-first is not offline-first* [online]. 2025-12. [cit. 2026-05-04]. Dostupné z: <https://shortdiv.com/posts/local-first-is-not-offline-first/>.
9. ALEX FEYERKE. *Designing Offline-First Web Apps* [online]. 2013-12. [cit. 2026-05-04]. Dostupné z: <https://alistapart.com/article/offline-first/>.
10. PETER VAN HARDENBERG. *#1 – PVH: An Intro to Local-First* [online]. [cit. 2026-05-04]. Dostupné z: https://youtu.be/0bjAI57_cDk.
11. SQLITE. *Long Term Support* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.sqlite.org/lts.html>.
12. SQLITE. *LoC Recommended Storage Format* [online]. 2018-05. [cit. 2026-05-04]. Dostupné z: <https://www.sqlite.org/locrsf.html>.

13. PETER VAN HARDENBERG. *Local First: the secret master plan* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.youtube.com/watch?v=9s80A08ggbM>.
14. LITT, GEOFFREY AND HOROWITZ, JOSH AND VAN HARDENBERG, PETER AND MATTHEWS, TODD. *Malleable Software: Restoring User Agency in a World of Locked-Down Apps* [online]. 2025-06. [cit. 2026-05-04]. Dostupné z: <https://www.inkandswitch.com/essay/malleable-software/>.
15. LOCAL-FIRST CONF. *Panel Discussion - The big questions of Local First* [online]. [cit. 2026-05-04]. Dostupné z: <https://youtu.be/86NmEerklTs>.
16. KYLE SIMPSON. *#17 – Kyle Simpson: Local-first identity* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.youtube.com/watch?v=jrCgiWKjdD0>.
17. M4HESHD. *better-sqlite3-multiple-ciphers – When is this library not appropriate?* [online]. [cit. 2026-05-04]. Dostupné z: <https://github.com/m4heshd/better-sqlite3-multiple-ciphers?tab=readme-ov-file#when-is-this-library-not-appropriate>.
18. RXDB. *Downsides of Local First / Offline First* [online]. [cit. 2026-05-04]. Dostupné z: <https://rxdb.info/downsides-of-offline-first.html>.
19. ROBIN SLOAN. *An app can be a home-cooked meal* [online]. 2020-02. [cit. 2026-05-04]. Dostupné z: <https://www.robinsloan.com/notes/home-cooked-app/>.
20. MAGGIE APPLETON. *Home-Cooked Software and Barefoot Developers* [online]. 2024-05. [cit. 2026-05-04]. Dostupné z: <https://maggieappleton.com/home-cooked-software>.
21. FACUNDO OLANO. *Software Possession for Personal Use* [online]. 2024-08. [cit. 2026-05-04]. Dostupné z: <https://olano.dev/blog/software-possession-for-personal-use/>.
22. INFOSAIC. *From Mainframes to the Cloud and Back Again: The Great Computing Loop We Can't Escape* [online]. 2025-01. [cit. 2026-05-04]. Dostupné z: <https://blog.infosaic.com/announcements/from-mainframes-to-the-cloud-and-back-again-the-great-computing-loop-we-cant-escape/>.
23. ARIF MOHAMED. *A history of cloud computing* [online]. 2018-04. [cit. 2026-05-04]. Dostupné z: <https://www.computerweekly.com/feature/A-history-of-cloud-computing>.
24. RAVI SINGH. *Complete History of IBM Lotus Notes to HCL Notes* [online]. 2025-08. [cit. 2026-05-04]. Dostupné z: <https://www.stellarinfo.com/blog/complete-history-ibm-lotus-notes-hcl-notes/>.
25. SOLID PROJECT. *About Solid* [online]. [cit. 2026-05-04]. Dostupné z: <https://solidproject.org/about>.

26. CHRISTIAN, REID AND DEORA, CHIRAAG. *Redefining Local-First Software: Decentralizing the Centralized Public Cloud* [online]. 2023-11. [cit. 2026-05-04]. Dostupné z: <https://medium.com/crv-insights/redefining-local-first-software-decentralizing-the-centralized-public-cloud-b1b89c200f18>.
27. JOERI SEBRECHTS. *The history of web application architecture* [online]. 2025-07. [cit. 2026-05-04]. Dostupné z: <https://plainvanillaweb.com/blog/articles/2025-07-13-history-architecture/>.
28. MCGRANAGHAN, MARK AND WIGGINS, ADAM. *Local-first software with Martin Kleppmann* [online]. 2021-10. [cit. 2026-05-04]. Dostupné z: <http://museapp.com/podcast/41-local-first-software/>.
29. TANTAMAN. *Why SQLite? Why Now?* [online]. 2022-08. [cit. 2026-05-04]. Dostupné z: <https://tantaman.com/2022-08-23-why-sqlite-why-now.html>.
30. SQLITE. *How SQLite Is Tested* [online]. [cit. 2026-05-04]. Dostupné z: <https://sqlite.org/testing.html>.
31. SQLITE. *Most Widely Deployed SQL Database Engine* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.sqlite.org/mostdeployed.html>.
32. SLICKER. *Modern SQLite: Features You Didn't Know It Had* [online]. [cit. 2026-05-04]. Dostupné z: <https://slicker.me/sqlite/features.htm>.
33. SYLVAIN KERKOUR. *Deep dive into Turso, the "SQLite rewrite in Rust"* [online]. 2026-01. [cit. 2026-05-04]. Dostupné z: <https://kerkour.com/turso-sqlite>.
34. DIVYA. *Local-first, why now?* [online]. 2025-12. [cit. 2026-05-04]. Dostupné z: <https://shortdiv.com/posts/local-first-why-now>.
35. THOMAS STEINER. *Deprecating and removing Web SQL* [online]. 2022-08. [cit. 2026-05-04]. Dostupné z: <https://developer.chrome.com/blog/deprecating-web-sql/>.
36. JAMES LONG. *A future for SQL on the web* [online]. 2021-08. [cit. 2026-05-04]. Dostupné z: <https://jlongster.com/future-sql-web>.
37. RALF KISTNER. *The Current State Of SQLite Persistence On The Web* [online]. 2025-11. [cit. 2026-05-04]. Dostupné z: <https://www.powersync.com/blog/sqlite-persistence-on-the-web>.
38. THOMAS STEINER. *SQLite Wasm in the browser backed by the Origin Private File System* [online]. 2023-01. [cit. 2026-05-04]. Dostupné z: <https://developer.chrome.com/blog/sqlite-wasm-in-the-browser-backed-by-the-origin-private-file-system>.
39. RXDB. *LocalStorage vs. IndexedDB vs. Cookies vs. OPFS vs. WASM-SQLite* [online]. 2026-04. [cit. 2026-05-04]. Dostupné z: <https://rxdb.info/articles/localstorage-indexeddb-cookies-opfs-sqlite-wasm.html>.

40. MOZILLA. *Storage quotas and eviction criteria* [online]. [cit. 2026-05-04]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria.
41. PATRICK BROSSET. *(Almost) everything about storing data on the web* [online]. 2023-01. [cit. 2026-05-04]. Dostupné z: <https://patrickbrosset.com/articles/2023-01-17-web-storage/>.
42. PETE LEPAGE. *Persistent storage* [online]. 2020-05. [cit. 2026-05-04]. Dostupné z: <https://web.dev/articles/persistent-storage>.
43. THOMAS STEINER. *The origin private file system* [online]. 2023-06. [cit. 2026-05-04]. Dostupné z: <https://web.dev/articles/origin-private-file-system>.
44. SANDRO MAGLIONE. *Local-first vs Sync engines* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.sandromaglione.com/articles/local-first-vs-sync-engines>.
45. FIATJAF. *nostr – Notes and Other Stuff Transmitted by Relays* [online]. 2020-11. [cit. 2026-05-04]. Dostupné z: <https://fiatjaf.com/nostr.html>.
46. STEVEN DEOBALD. *LocalFirst: You Keep Using That Word* [online]. 2026-01. [cit. 2026-05-04]. Dostupné z: <https://www.deobald.ca/essays/2026-01-01-localfirst-you-keep-using-that-word/>.
47. SHAPIRO, Marc; PREGUIÇA, Nuno; BAQUERO, Carlos; ZAWIRSKI, Marek. A comprehensive study of Convergent and Commutative Replicated Data Types. *INRIA Research Report*. 2011. Dostupné také z: <https://inria.hal.science/hal-00932836v1/document>.
48. MARTIN KLEPPMANN. *CRDTs: The Hard Parts* [online]. [cit. 2026-05-04]. Dostupné z: <https://youtu.be/x7drE24geUw>.
49. MATTHEW WEIDNER. *Collaborative Text Editing without CRDTs or OT* [online]. 2025-05. [cit. 2026-05-04]. Dostupné z: <https://mattweidner.com/2025/05/21/text-without-crdts.html>.
50. BARTOSZ SYPYTKOWSKI. *Operation-based CRDTs: arrays (part 1)* [online]. 2020-09. [cit. 2026-05-04]. Dostupné z: <https://www.bartoszsypytkowski.com/operation-based-crdts-arrays-1/>.
51. JOSEPH GENTLE. *I was wrong. CRDTs are the future* [online]. 2020-09. [cit. 2026-05-04]. Dostupné z: <https://josephg.com/blog/crdts-are-the-future/>.
52. KULKARNI, Sandeep S.; DEMIRBAS, Murat; MADAPPA, Deepak; AVVA, Bharadwaj; LEONE, Marcelo. Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases. *SUNY Buffalo Tech Report 2014-04*. 2014. Dostupné také z: <https://cse.buffalo.edu/tech-reports/2014-04.pdf>.
53. JARED FORSYTH. *Hybrid Logical Clocks* [online]. 2020-02. [cit. 2026-05-04]. Dostupné z: <https://jaredforsyth.com/posts/hybrid-logical-clocks/>.

54. MEYER, Aljoscha. Range-Based Set Reconciliation. *arXiv:2212.13567*. 2022. Dostupné také z: <https://arxiv.org/pdf/2212.13567>.
55. BELLARE, Mihir; GOLDREICH, Oded; GOLDWASSER, Shafi. Incremental Cryptography: The Case of Hashing and Signing. *CRYPTO*. 1994. Dostupné také z: <https://cseweb.ucsd.edu/~mihir/papers/inc-hash.pdf>.
56. HERB CAUDILL. *Alice and Bob in Wonderland* [online]. 2024-06. [cit. 2026-05-04]. Dostupné z: <https://herbcaudill.com/words/20240602-local-first-auth>.
57. PHIL ZIMMERMANN. *Why I Wrote PGP* [online]. 1999. [cit. 2026-05-04]. Dostupné z: <https://groups.csail.mit.edu/mac/classes/6.805/articles/crypto/cypherpunks/zimmermann-why-pgp.html>.
58. STEVEN LEVY. *Why Are We Fighting the Crypto Wars Again?* [online]. 2016-03. [cit. 2026-05-04]. Dostupné z: <https://medium.com/backchannel/why-are-we-fighting-the-crypto-wars-again-b5310a423295>.
59. OWNID. *What Are Passkeys? What Is a Passkey? Passkey Authentication Explained* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.passkeys.com/what-are-passkeys>.
60. COOLAJ86 AND GETIFY. *Thoughts & Notes on rawId vs userHandle as shared secret/seed* [online]. 2024-08. [cit. 2026-05-04]. Dostupné z: <https://github.com/mylofi/local-data-lock/discussions/8>.
61. GETIFY. *Investigate using PRF authenticator extension* [online]. 2024-08. [cit. 2026-05-04]. Dostupné z: <https://github.com/mylofi/local-data-lock/issues/2>.
62. MOZILLA. *[meta] Support WebAuthn PRF extension* [online]. 2023-08. [cit. 2026-05-04]. Dostupné z: https://bugzilla.mozilla.org/show_bug.cgi?id=1863819.
63. W3C. *Web Authentication: Large blob storage extension (largeBlob)* [online]. 2026-01. [cit. 2026-05-04]. Dostupné z: <https://www.w3.org/TR/webauthn-3/#sctn-large-blob-extension>.
64. BARNES, RICHARD AND BEURDOUCHE, BENJAMIN AND ROBERT, RAPHAEL AND MILLICAN, JON AND OMARA, EMAD AND COHN-GORDON, KATRIEL. *The Messaging Layer Security (MLS) Protocol (RFC 9420)* [online]. 2023-07. [cit. 2026-05-04]. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc9420>.
65. KONRAD KOHBROK. *Decentralized Messaging Layer Security* [online]. 2025-10. [cit. 2026-05-04]. Dostupné z: <https://phnx-im.github.io/dmls-spec/draft-kohbrok-mls-dmls.html>.
66. GOOD, ALEX AND MUMM, JOHN AND ZELENKA, BROOKLYN. *Keyhive: Local-first Access Control* [online]. 2024. [cit. 2026-05-04]. Dostupné z: <https://www.inkandswitch.com/keyhive/notebook/>.

67. IETF. *Messaging layer security: Encrypting a group chat* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.youtube.com/watch?v=FESp2LHd42U>.
68. DOUG HOYTE. *Range-Based Set Reconciliation* [online]. 2023-12. [cit. 2026-05-04]. Dostupné z: <https://logperiodic.com/rbsr.html>.
69. ABU-GHAZALEH, NAEL AND PONOMAREV, DMITRY AND EVTYUSHKIN, DMITRY. *How the Spectre and Meltdown Hacks Really Worked* [online]. 2019-02. [cit. 2026-05-04]. Dostupné z: <https://spectrum.ieee.org/how-the-spectre-and-meltdown-hacks-really-worked>.
70. LUKE WAGNER. *Mitigations landing for new class of timing attack* [online]. 2018-01. [cit. 2026-05-04]. Dostupné z: <https://blog.mozilla.org/security/2018/01/03/mitigations-landing-new-class-timing-attack/>.
71. SCOTT TOLINSKI. *The Spectrum of Local First Libraries* [online]. [cit. 2026-05-04]. Dostupné z: <https://tolin.ski/posts/local-first-options>.
72. SCHICKLING, JOHANNES AND MARTIN, JESS. *Local-First Landscape* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.localfirst.fm/landscape>.
73. EVOLU. *Documentation* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.evolu.dev/docs>.
74. JOSH GOLDBERG. *Branded Types* [online]. 2024-08. [cit. 2026-05-04]. Dostupné z: <https://www.learningtypescript.com/articles/branded-types>.
75. EVOLU. *Interface: Task* [online]. [cit. 2026-05-04]. Dostupné z: <https://www.evolu.dev/docs/api-reference/common/Task/interfaces/Task>.
76. GRAPHQL. *Schema Design* [online]. [cit. 2026-05-04]. Dostupné z: <https://graphql.org/learn/schema-design/>.
77. SQLITE. *Database Remote-Copy Tool For SQLite* [online]. [cit. 2026-05-04]. Dostupné z: <https://sqlite.org/rsync.html>.
78. DANIEL STEIGERWALD. *Scaling local-first software* [online]. 2025-05. [cit. 2026-05-04]. Dostupné z: <https://www.evolu.dev/blog/scaling-local-first-software>.
79. BARMAN, L. AND NIKITIN, K. AND LUEKS, W. AND FORD, B. AND HUBAUX, J.-P. *Padmé: Efficiently Hiding File Sizes* [online]. 2020-08. [cit. 2026-05-04]. Dostupné z: <https://lbarman.ch/blog/padme/>.
80. DANIEL LEMIRE. *Are 64-bit random identifiers free from collision?* [online]. 2019-12. [cit. 2026-05-04]. Dostupné z: <https://lemire.me/blog/2019/12/12/are-64-bit-random-identifiers-free-from-collision/>.
81. EVOLU. *Privacy* [online]. 2026. [cit. 2026-05-04]. Dostupné z: <https://www.evolu.dev/docs/privacy>.
82. GORDON BRANDER. *Credible exit* [online]. 2022-04. [cit. 2026-05-04]. Dostupné z: <https://newsletter.squishy.computer/p/credible-exit>.

A Odkaz na zdrojový kód

- Zdrojové kódy aplikace (aktuální ke dni 11. 5. 2026) si lze zobrazit zde:
<https://github.com/arbyys/tul-dp>